

**LAPORAN AKHIR**  
**PROYEK PRAKTIKUM IoT**  
**PERANCANGAN SISTEM *EARLY WARNING***  
**GANGGUAN INFRASTRUKTUR BANK BERBASIS IOT**



**Disusun Oleh:**

|                |                                           |                      |
|----------------|-------------------------------------------|----------------------|
| Nama (NIM)     | : Nur Halimah                             | (24/540349/SV/24822) |
|                | Tanaya Nindya Pradipta                    | (24/536017/SV/24365) |
|                | Insyania Cahya Wiguna                     | (24/533789/SV/23938) |
|                | Annas Meilya Khodijah                     | (24/544888/SV/25557) |
| Kelas          | : RI4A2                                   |                      |
| Dosen Pengampu | : Ardhi Wicaksono Santoso., S.Kom., M.Cs. |                      |

**SARJANA TERAPAN TEKNOLOGI REKAYASA INTERNET**  
**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**  
**SEKOLAH VOKASI**  
**UNIVERSITAS GADJAH MADA**  
**YOGYAKARTA**  
**2026**

# DAFTAR ISI

|                                             |          |
|---------------------------------------------|----------|
| <b>DAFTAR ISI .....</b>                     | <b>1</b> |
| <b>DAFTAR GAMBAR.....</b>                   | <b>4</b> |
| <b>DAFTAR TABEL .....</b>                   | <b>5</b> |
| <b>A. PENDAHULUAN.....</b>                  | <b>6</b> |
| 1.1 Latar Belakang .....                    | 6        |
| 1.2 Rumusan Masalah .....                   | 7        |
| 1.3 Tujuan.....                             | 7        |
| 1.4 Manfaat .....                           | 8        |
| 1. Manfaat Akademis.....                    | 8        |
| 2. Manfaat Praktis .....                    | 8        |
| 1.5 Batasan Masalah.....                    | 8        |
| <b>B. TINJAUAN PUSTAKA .....</b>            | <b>9</b> |
| 2.1 Internet of Things (IoT).....           | 9        |
| 2.2 Sensor dan Aktuator yang Digunakan..... | 9        |
| 2.2.1 Sensor Suhu dan Kelembaban .....      | 9        |
| 2.2.2 Sensor Getaran.....                   | 10       |
| 2.2.3 Sensor Pintu .....                    | 10       |
| 2.2.4 Aktuator LED dan Buzzer .....         | 10       |
| 2.2.5 RFID RC522 .....                      | 10       |
| 2.2.6 Motor Servo SG90 .....                | 11       |
| 2.3 Platform IoT .....                      | 11       |
| 2.3.1 MQTT.....                             | 11       |
| 2.3.2 Node-RED.....                         | 11       |
| 2.3.3 Grafana .....                         | 12       |
| 2.3.4 Virtual Private Server (VPS) .....    | 12       |
| 2.3.5 Apache Hadoop HDFS .....              | 12       |
| 2.4 Studi Literatur Proyek Serupa .....     | 13       |
| 2.5 Landasan Teori Terkait .....            | 13       |
| 2.5.1 Komunikasi Data pada IoT.....         | 13       |
| 2.5.2 Mikrokontroler ESP32.....             | 13       |

|                                                   |    |
|---------------------------------------------------|----|
| 2.5.3 Protokol MQTT .....                         | 13 |
| 2.5.4 Monitoring dan Early Warning System.....    | 14 |
| <b>C. PERANCANGAN SISTEM</b> .....                | 15 |
| 3.1 Desain Sistem IoT .....                       | 15 |
| 3.1.1 Diagram Blok .....                          | 15 |
| 3.1.2 Arsitektur Sistem .....                     | 17 |
| 3.2 Alat dan Bahan.....                           | 19 |
| 3.3 Diagram Alur Kerja (Flowchart) .....          | 21 |
| 3.4 Desain Perangkat Keras (Hardware) .....       | 22 |
| 3.4.1 I/O Mapping .....                           | 22 |
| 3.4.2 Pemrograman Hardware .....                  | 24 |
| 3.5 Desain Perangkat Lunak (Software) .....       | 24 |
| 3.5.1 Algoritma utama .....                       | 24 |
| 3.5.2 Bahasa pemrograman/platform .....           | 26 |
| 3.5.3 Interface aplikasi .....                    | 26 |
| 3.6 Desain Dashboard (Software) .....             | 27 |
| <b>D. IMPLEMENTASI DAN PENGUJIAN</b> .....        | 29 |
| 4.1 Implementasi Rangkaian dan Program.....       | 29 |
| 4.1.1 Data Center .....                           | 29 |
| 4.1.2 Aktuator .....                              | 29 |
| 4.1.3 Vault.....                                  | 30 |
| 4.2 Konfigurasi Platform IoT .....                | 31 |
| 4.2.1 Konfigurasi MQTT Broker.....                | 31 |
| 4.2.2 Konfigurasi Node-RED .....                  | 31 |
| 4.2.3 Konfigurasi Penyimpanan Hadoop (HDFS) ..... | 31 |
| 4.2.4 Diagram Alur Konfigurasi Platform .....     | 32 |
| 4.3 Pengujian Sistem .....                        | 33 |
| 4.3.1 Pengujian Sensor.....                       | 33 |
| 4.3.2 Pengujian Jaringan.....                     | 35 |
| 4.3.3 Pengujian Kinerja Aplikasi .....            | 35 |
| 4.4 Analisis Hasil Pengujian .....                | 37 |
| <b>E. PENUTUP</b> .....                           | 39 |
| 5.1 Kesimpulan.....                               | 39 |

|                            |           |
|----------------------------|-----------|
| 5.2 Saran.....             | 39        |
| <b>DAFTAR PUSTAKA.....</b> | <b>40</b> |
| <b>LAMPIRAN .....</b>      | <b>41</b> |

## DAFTAR GAMBAR

|                                            |    |
|--------------------------------------------|----|
| Gambar 1. Diagram Blok .....               | 15 |
| Gambar 2. Arsitektur Sistem.....           | 17 |
| Gambar 3. Flowchart Sistem.....            | 21 |
| Gambar 4. Peta Wiring MCU + DHT11 .....    | 29 |
| Gambar 5. Peta Wiring MCU + Aktuator ..... | 29 |
| Gambar 6. Peta Wiring MCU + Vault.....     | 30 |

## DAFTAR TABEL

|                                      |    |
|--------------------------------------|----|
| Table 1. Alat dan Bahan .....        | 19 |
| Table 2. Pin MCU + Data Center ..... | 22 |
| Table 3. Pin MCU + Vault.....        | 23 |
| Table 4. Pin MCU + Aktuator.....     | 23 |
| Table 5. Bahasa Pemrograman.....     | 26 |

## A. PENDAHULUAN

### 1.1 Latar Belakang

Perkembangan teknologi Internet of Things (IoT) telah membuka peluang besar dalam membangun sistem pemantauan yang mampu bekerja secara otomatis dan real-time. IoT memungkinkan berbagai perangkat fisik seperti sensor, aktuator, dan mikrokontroler saling terhubung melalui jaringan internet untuk mengumpulkan, mengirimkan, serta mengolah data secara berkelanjutan [1]. Pemanfaatan IoT saat ini telah diterapkan pada berbagai sektor, termasuk industri perbankan yang membutuhkan tingkat keamanan dan keandalan infrastruktur yang tinggi [2].

Dalam operasional perbankan modern, keberlangsungan layanan sangat bergantung pada kondisi infrastruktur pendukung seperti ruang server (*data center*), mesin ATM, serta sistem keamanan penyimpanan aset. Gangguan berupa peningkatan suhu ruang server, getaran yang tidak normal, maupun akses tidak sah terhadap area tertentu dapat menyebabkan kerusakan perangkat, gangguan layanan, hingga risiko keamanan yang berdampak pada operasional bank. Oleh karena itu, diperlukan suatu sistem yang mampu melakukan pemantauan kondisi infrastruktur secara terus-menerus dan memberikan peringatan dini ketika terjadi kondisi yang berpotensi menimbulkan gangguan [3].

Salah satu solusi yang dapat diterapkan adalah pengembangan sistem *early warning* berbasis Internet of Things. Sistem ini memanfaatkan sensor suhu dan kelembaban (DHT11), sensor getaran, sensor pintu, serta teknologi RFID untuk autentikasi akses yang dihubungkan dengan mikrokontroler ESP32. Data yang diperoleh kemudian dikirimkan ke server menggunakan protokol MQTT. Selain itu, motor servo digunakan sebagai mekanisme buka-tutup pintu brankas secara otomatis setelah proses autentikasi berhasil dilakukan [4]. Apabila terdeteksi kondisi yang melebihi ambang batas yang telah ditentukan, sistem dapat memberikan peringatan melalui aktuator maupun dashboard monitoring sehingga respons terhadap gangguan dapat dilakukan lebih cepat [2].

Otoritas Jasa Keuangan (OJK) melalui Panduan Resiliensi Digital bagi Bank Umum juga menekankan pentingnya kemampuan lembaga perbankan dalam mengantisipasi, mendeteksi, dan merespons gangguan teknologi informasi guna menjaga keberlangsungan layanan dan keamanan operasional [5]. Oleh karena itu, penerapan sistem monitoring dan *early warning* berbasis IoT menjadi salah satu pendekatan yang relevan untuk mendukung ketahanan infrastruktur perbankan.

Pada proyek ini telah dilakukan implementasi awal berupa pembuatan prototype infrastruktur perbankan berbahan akrilik yang merepresentasikan area data center dan brankas (vault). Selain itu, pemasangan sensor suhu dan kelembaban (DHT11), sensor getaran (SW420), sensor pintu (MC38), RFID RC522, LED, buzzer, dan motor servo SG90

telah dilakukan serta diintegrasikan dengan ESP32 untuk mendukung proses monitoring dan keamanan secara real-time. Hasil pengujian awal menunjukkan bahwa data sensor berhasil dikirim menuju server untuk diproses lebih lanjut.

Berdasarkan kondisi tersebut, proyek ini bertujuan untuk merancang dan mengimplementasikan sistem *Early Warning Gangguan Infrastruktur Bank Berbasis IoT* yang mampu melakukan pemantauan kondisi lingkungan secara real-time, mendeteksi potensi gangguan lebih awal, serta memberikan notifikasi secara cepat guna meningkatkan keamanan dan keandalan operasional infrastruktur perbankan.

## **1.2 Rumusan Masalah**

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang sistem monitoring infrastruktur perbankan berbasis Internet of Things (IoT) yang mampu melakukan pengambilan data secara real-time?
2. Bagaimana mengintegrasikan sensor suhu dan kelembaban, sensor getaran, sensor pintu, RFID, serta motor servo dengan mikrokontroler ESP32 untuk mendukung monitoring dan keamanan infrastruktur perbankan?
3. Bagaimana mengirimkan data hasil pembacaan sensor ke server menggunakan protokol MQTT?
4. Bagaimana merancang sistem peringatan dini (*early warning system*) yang mampu memberikan notifikasi ketika terjadi gangguan pada infrastruktur yang dipantau?
5. Bagaimana mengevaluasi kinerja sistem IoT yang dikembangkan dalam mendukung monitoring infrastruktur perbankan?

## **1.3 Tujuan**

Tujuan yang ingin dicapai dalam proyek ini adalah:

1. Merancang dan membangun sistem monitoring infrastruktur perbankan berbasis Internet of Things (IoT).
2. Mengimplementasikan sensor suhu dan kelembaban (DHT11), sensor getaran, sensor pintu, RFID, dan motor servo sebagai perangkat pemantauan kondisi lingkungan serta keamanan akses.
3. Mengembangkan sistem komunikasi data menggunakan ESP32 dan protokol MQTT untuk pengiriman data secara real-time.
4. Membangun sistem *early warning* yang mampu memberikan peringatan ketika terdeteksi kondisi abnormal.
5. Menguji dan mengevaluasi kemampuan sistem dalam melakukan monitoring dan deteksi dini terhadap gangguan infrastruktur.



## **1.4 Manfaat**

Manfaat yang diharapkan dari pelaksanaan proyek ini adalah:

### **1. Manfaat Akademis**

- Menambah pemahaman mengenai implementasi Internet of Things pada sistem monitoring dan keamanan.
- Menjadi sarana penerapan teori yang diperoleh pada mata kuliah Praktikum Internet of Things ke dalam bentuk proyek nyata.
- Menjadi referensi bagi pengembangan penelitian dan proyek serupa pada masa mendatang.

### **2. Manfaat Praktis**

- Membantu proses pemantauan kondisi infrastruktur secara real-time.
- Mempercepat proses deteksi gangguan melalui sistem peringatan dini.
- Meningkatkan keamanan lingkungan yang dipantau melalui penggunaan sensor dan notifikasi otomatis.
- Mengurangi risiko kerusakan perangkat akibat keterlambatan penanganan gangguan.

## **1.5 Batasan Masalah**

Agar pembahasan lebih terarah, proyek ini memiliki beberapa batasan sebagai berikut:

1. Sistem dikembangkan dalam bentuk prototype yang merepresentasikan infrastruktur perbankan berupa area data center, ATM, dan brankas.
2. Mikrokontroler yang digunakan adalah ESP32 sebagai pusat pengendali perangkat IoT.
3. Sensor yang digunakan terdiri dari sensor suhu dan kelembaban (DHT11), sensor getaran (SW420), sensor pintu (MC38), dan RFID RC522 untuk autentikasi akses.
4. Aktuator yang digunakan berupa LED dan buzzer sebagai indikator peringatan lokal serta motor servo SG90 sebagai mekanisme buka-tutup pintu brankas.
5. Komunikasi data dilakukan menggunakan protokol MQTT melalui jaringan internet.
6. Sistem berfokus pada fungsi monitoring dan pemberian peringatan dini, serta tidak mencakup pengendalian otomatis terhadap perangkat yang dipantau.
7. Sistem tidak terintegrasi secara langsung dengan sistem inti perbankan (*core banking system*).
8. Pengujian dilakukan pada lingkungan prototype dan belum diterapkan pada infrastruktur perbankan yang sebenarnya.

## B. TINJAUAN PUSTAKA

### 2.1 Internet of Things (IoT)

Internet of Things (IoT) merupakan paradigma yang memungkinkan berbagai objek fisik terhubung ke jaringan internet sehingga mampu mengumpulkan, mengirimkan, dan bertukar data secara otomatis tanpa intervensi manusia secara langsung [1]. Konsep IoT pertama kali diperkenalkan oleh Kevin Ashton pada tahun 1999 dan terus berkembang seiring meningkatnya kebutuhan sistem otomatisasi dan monitoring pada berbagai sektor industri [6].

Dalam implementasinya, IoT terdiri atas beberapa komponen utama yaitu perangkat sensor, aktuator, mikrokontroler, jaringan komunikasi, serta platform pengolahan data. Sensor berfungsi untuk memperoleh informasi dari lingkungan, sedangkan aktuator digunakan untuk memberikan respons terhadap kondisi tertentu. Data yang diperoleh kemudian dikirim melalui jaringan menuju server atau platform cloud untuk diproses dan dianalisis [2].

Pada proyek ini, IoT digunakan untuk memantau kondisi infrastruktur perbankan melalui sensor suhu, sensor getaran, dan sensor pintu yang terhubung ke mikrokontroler ESP32. Data hasil pembacaan sensor dikirimkan secara real-time ke server menggunakan protokol MQTT sehingga kondisi sistem dapat dipantau secara berkelanjutan.

Arsitektur IoT secara umum terdiri atas empat lapisan utama yaitu *perception layer*, *network layer*, *processing layer*, dan *application layer* [2]. Pada proyek ini, sensor dan ESP32 berperan sebagai *perception layer*, jaringan komunikasi dan MQTT sebagai *network layer*, Node-RED dan VPS sebagai *processing layer*, sedangkan Grafana berperan sebagai *application layer*. Selain itu, Apache Hadoop HDFS digunakan sebagai lapisan penyimpanan data historis (*data storage layer*) yang mendukung penyimpanan dan backup data monitoring dalam jangka panjang.

### 2.2 Sensor dan Aktuator yang Digunakan

#### 2.2.1 Sensor Suhu dan Kelembaban

Sensor suhu merupakan perangkat yang digunakan untuk mengukur temperatur lingkungan dan mengubahnya menjadi sinyal listrik yang dapat diproses oleh mikrokontroler. Monitoring suhu sangat penting pada lingkungan data center karena peningkatan suhu yang berlebihan dapat menyebabkan penurunan performa perangkat bahkan kerusakan sistem [7].

Dalam proyek ini, sensor DHT11 digunakan untuk memantau suhu dan kelembaban pada area data center. Data yang diperoleh dikirim secara berkala ke server untuk dianalisis dan digunakan sebagai dasar pemberian peringatan dini ketika suhu atau kelembaban melebihi batas yang telah ditentukan.

### **2.2.2 Sensor Getaran**

Sensor getaran digunakan untuk mendeteksi adanya guncangan, benturan, atau aktivitas fisik yang tidak normal pada suatu objek. Sensor ini banyak digunakan dalam sistem keamanan dan pemantauan kondisi peralatan karena mampu mendeteksi perubahan mekanis yang berpotensi menimbulkan gangguan [8].

Pada proyek ini, sensor getaran dipasang pada area yang merepresentasikan mesin ATM dan brankas untuk mendeteksi aktivitas mencurigakan seperti benturan atau upaya pembongkaran.

### **2.2.3 Sensor Pintu**

Sensor pintu berfungsi untuk mendeteksi kondisi terbuka atau tertutupnya suatu pintu. Sensor ini umum digunakan pada sistem keamanan karena mampu memberikan informasi terkait akses ke area tertentu [9].

Dalam implementasi proyek, sensor pintu digunakan untuk memantau akses ke area brankas sehingga sistem dapat memberikan notifikasi ketika terjadi pembukaan pintu yang tidak sesuai dengan kondisi normal.

### **2.2.4 Aktuator LED dan Buzzer**

Aktuator merupakan perangkat yang digunakan untuk memberikan respons terhadap hasil pemrosesan data. Pada proyek ini digunakan LED sebagai indikator visual dan buzzer sebagai indikator suara ketika sistem mendeteksi kondisi abnormal [2].

LED digunakan untuk menunjukkan status sistem secara langsung, sedangkan buzzer berfungsi memberikan peringatan suara sehingga pengguna dapat segera mengetahui adanya gangguan tanpa harus melihat dashboard monitoring.

### **2.2.5 RFID RC522**

Radio Frequency Identification (RFID) merupakan teknologi identifikasi otomatis yang memanfaatkan gelombang radio untuk mengenali objek atau pengguna tanpa kontak fisik secara langsung. Sistem RFID umumnya terdiri atas tag, reader, dan mikrokontroler sebagai pengolah data. Ketika tag RFID didekatkan ke reader, informasi identitas yang tersimpan pada tag akan dibaca dan dikirimkan ke sistem untuk proses autentikasi.

Pada proyek ini digunakan modul RFID RC522 yang terhubung dengan ESP32 untuk melakukan autentikasi akses ke area brankas. Sistem akan membandingkan UID kartu yang terbaca dengan daftar UID yang telah terdaftar. Jika UID sesuai, maka akses diberikan dan pintu dapat dibuka. Sebaliknya, apabila UID tidak dikenali, sistem akan memberikan peringatan dan menolak akses. RFID dipilih karena memiliki proses

autentikasi yang cepat, biaya implementasi rendah, serta mudah diintegrasikan dengan ESP32 melalui antarmuka SPI. [15].

### **2.2.6 Motor Servo SG90**

Motor servo merupakan aktuator yang mampu menghasilkan gerakan sudut secara presisi berdasarkan sinyal Pulse Width Modulation (PWM) yang diberikan oleh mikrokontroler. Berbeda dengan motor DC biasa, servo dapat dikendalikan untuk bergerak ke posisi tertentu sesuai kebutuhan sistem.

Pada proyek ini digunakan motor servo SG90 sebagai mekanisme buka-tutup pintu brankas. Servo dikendalikan oleh ESP32 dan akan bergerak ke posisi terbuka ketika proses autentikasi RFID berhasil dilakukan. Setelah batas waktu akses berakhir, servo akan kembali ke posisi semula untuk menutup pintu secara otomatis. Servo SG90 dipilih karena memiliki ukuran kecil, konsumsi daya rendah, dan mampu menghasilkan gerakan sudut yang cukup untuk mensimulasikan mekanisme buka-tutup pintu pada prototype [16].

## **2.3 Platform IoT**

### **2.3.1 MQTT**

Message Queuing Telemetry Transport (MQTT) merupakan protokol komunikasi berbasis *publish-subscribe* yang dirancang untuk perangkat dengan sumber daya terbatas dan jaringan dengan bandwidth rendah [4]. MQTT memiliki ukuran paket data yang kecil sehingga sangat cocok digunakan pada sistem IoT.

Pada arsitektur MQTT terdapat tiga komponen utama yaitu *publisher*, *broker*, dan *subscriber*. Dalam proyek ini, ESP32 bertindak sebagai *publisher* yang mengirimkan data sensor, Mosquitto MQTT berfungsi sebagai *broker*, sedangkan Node-RED bertindak sebagai *subscriber* yang menerima data untuk diproses.

### **2.3.2 Node-RED**

Node-RED merupakan platform pemrograman berbasis *flow* yang dikembangkan oleh IBM untuk memudahkan integrasi perangkat keras, layanan web, dan aplikasi IoT [10]. Platform ini menyediakan antarmuka visual sehingga proses pengembangan sistem dapat dilakukan tanpa menulis kode program secara kompleks.

Dalam proyek ini, Node-RED digunakan untuk menerima data dari MQTT broker, melakukan pemrosesan data, menentukan kondisi normal atau abnormal, serta mengirimkan notifikasi kepada pengguna.

### **2.3.3 Grafana**

Grafana merupakan platform visualisasi data berbasis web yang banyak digunakan untuk monitoring sistem dan analisis data real-time [11]. Grafana mampu menampilkan data dalam bentuk grafik, tabel, maupun indikator sehingga memudahkan pengguna dalam memahami kondisi sistem.

Pada proyek ini, Grafana digunakan untuk menampilkan data sensor yang diterima dari perangkat IoT sehingga administrator dapat memantau kondisi sistem secara real-time.

### **2.3.4 Virtual Private Server (VPS)**

Virtual Private Server (VPS) merupakan layanan server virtual yang berjalan pada lingkungan komputasi berbasis cloud dan dapat diakses melalui jaringan internet [12]. VPS memungkinkan pengguna menjalankan berbagai layanan jaringan secara mandiri dengan sumber daya yang terisolasi, sehingga lebih fleksibel dibandingkan layanan hosting konvensional.

Pada proyek ini, VPS digunakan sebagai server untuk menjalankan layanan Mosquitto MQTT Broker yang berfungsi sebagai pusat komunikasi antar perangkat IoT. MQTT Broker menerima data yang dipublikasikan oleh perangkat ESP32 dan mendistribusikannya kepada seluruh subscriber yang terdaftar. Sementara itu, layanan Node-RED, Grafana, dan Apache Hadoop HDFS dijalankan pada VM Ubuntu lokal yang digunakan sebagai lingkungan pengembangan, pemrosesan data, visualisasi monitoring, dan penyimpanan data historis.

### **2.3.5 Apache Hadoop HDFS**

Apache Hadoop Distributed File System (HDFS) merupakan sistem penyimpanan terdistribusi yang dikembangkan dalam ekosistem Apache Hadoop untuk menangani penyimpanan data dalam jumlah besar secara andal dan efisien. HDFS membagi data menjadi beberapa blok dan menyimpannya secara terdistribusi sehingga meningkatkan ketersediaan data serta mendukung mekanisme backup dan pemrosesan data skala besar.

Pada proyek ini, HDFS digunakan sebagai media penyimpanan data monitoring yang diterima dari perangkat IoT melalui MQTT dan Node-RED. Data sensor yang tersimpan dapat digunakan sebagai arsip historis, bahan analisis performa sistem, serta cadangan data apabila terjadi gangguan pada sistem monitoring utama. Implementasi HDFS juga mendukung mekanisme snapshot berkala dan backup data untuk meningkatkan keandalan penyimpanan.

## 2.4 Studi Literatur Proyek Serupa

Penelitian oleh Al-Fuqaha dkk. [2] menjelaskan bahwa IoT dapat digunakan sebagai sistem monitoring real-time melalui integrasi sensor, jaringan komunikasi, dan platform pengolahan data. Penelitian tersebut menunjukkan bahwa penggunaan protokol ringan seperti MQTT mampu meningkatkan efisiensi komunikasi pada perangkat IoT.

Penelitian yang dilakukan oleh Komninos dkk. [3] membahas penerapan sensor dan sistem monitoring pada lingkungan yang membutuhkan keamanan tinggi. Hasil penelitian menunjukkan bahwa integrasi sensor dan sistem notifikasi mampu meningkatkan kemampuan deteksi dini terhadap gangguan keamanan.

Penelitian lain yang dilakukan oleh Bandyopadhyay dan Sen [13] menunjukkan bahwa sistem monitoring berbasis IoT dapat digunakan untuk mendeteksi kondisi lingkungan secara berkelanjutan dan memberikan respons secara otomatis ketika terjadi perubahan parameter yang signifikan.

Berdasarkan berbagai penelitian tersebut dapat disimpulkan bahwa sistem monitoring berbasis IoT memiliki potensi yang besar untuk diterapkan pada infrastruktur perbankan. Namun, sebagian besar penelitian hanya berfokus pada aspek monitoring lingkungan atau keamanan secara terpisah. Oleh karena itu, proyek ini dikembangkan dengan mengintegrasikan pemantauan suhu, getaran, dan akses pintu dalam satu sistem *early warning* yang terhubung dengan dashboard monitoring dan notifikasi real-time.

## 2.5 Landasan Teori Terkait

### 2.5.1 Komunikasi Data pada IoT

Komunikasi data merupakan proses pertukaran informasi antara perangkat pengirim dan perangkat penerima melalui media transmisi tertentu. Pada sistem IoT, komunikasi data berperan penting dalam menghubungkan sensor, mikrokontroler, server, dan aplikasi monitoring [2].

### 2.5.2 Mikrokontroler ESP32

ESP32 merupakan mikrokontroler yang dikembangkan oleh Espressif Systems dan telah dilengkapi dengan modul Wi-Fi serta Bluetooth dalam satu chip [8]. Kemampuan tersebut menjadikan ESP32 sangat populer untuk implementasi sistem IoT karena mampu melakukan komunikasi data secara langsung melalui jaringan internet.

### 2.5.3 Protokol MQTT

MQTT menggunakan model komunikasi *publish-subscribe* yang berbeda dengan model *request-response* pada HTTP. Pada model MQTT, perangkat publisher

mengirimkan data ke broker, kemudian broker mendistribusikan data tersebut kepada subscriber yang berlangganan pada topik tertentu [4].

Keunggulan MQTT meliputi penggunaan bandwidth yang rendah, konsumsi daya yang kecil, serta kemampuan komunikasi real-time sehingga cocok digunakan pada perangkat IoT [4].

#### **2.5.4 Monitoring dan Early Warning System**

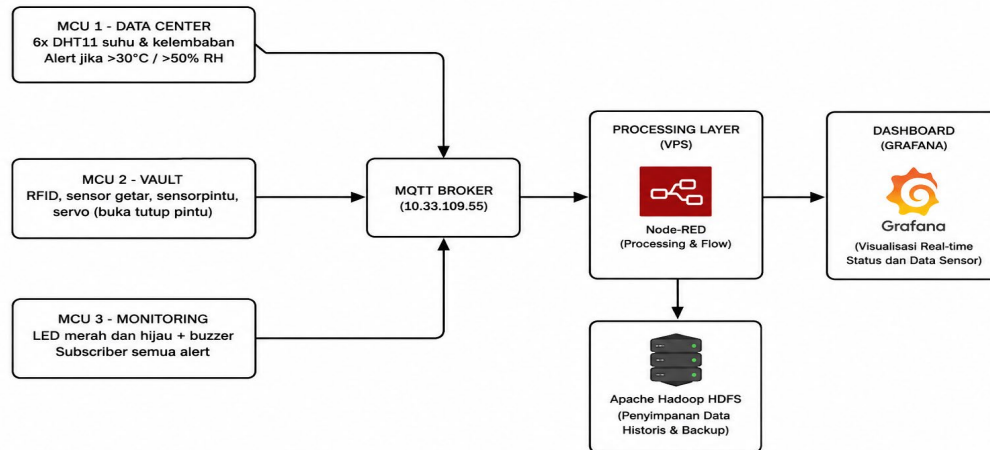
Monitoring merupakan proses pengumpulan dan pengamatan data secara berkelanjutan untuk mengetahui kondisi suatu sistem. Sementara itu, *Early Warning System* (EWS) merupakan mekanisme yang digunakan untuk mendeteksi potensi gangguan sejak dini sehingga tindakan pencegahan dapat dilakukan sebelum terjadi kerusakan yang lebih besar [14].

Pada proyek ini, konsep EWS diterapkan melalui analisis data sensor suhu, getaran, dan sensor pintu yang kemudian menghasilkan notifikasi ketika nilai tertentu melebihi ambang batas yang telah ditetapkan.

## C. PERANCANGAN SISTEM

### 3.1 Desain Sistem IoT

#### 3.1.1 Diagram Blok



Gambar 1. Diagram Blok

Sistem Early Warning Infrastruktur Bank ini dirancang menggunakan arsitektur IoT berbasis protokol MQTT dengan tiga komponen utama: perangkat sensor (publisher), broker MQTT, dan dashboard monitoring. Komponen sistem dari arsitektur project yaitu:

#### 1. MCU 1 - Data Center

MCU 1 berfungsi sebagai publisher data lingkungan ruang server. Perangkat ini dilengkapi dengan enam sensor DHT11 yang dipasang pada dinding untuk memantau suhu dan kelembaban secara real-time. Sistem akan menghasilkan peringatan otomatis apabila suhu melebihi 30°C atau kelembaban melebihi 50%. Data yang dikumpulkan kemudian dipublikasikan ke MQTT broker melalui beberapa topik, yaitu data suhu dan kelembaban per sensor, rata-rata keseluruhan, serta status alert.

#### 2. MCU 2 - Vault

MCU 2 bertugas mengamankan dan memantau ruang brankas. Perangkat ini mengintegrasikan beberapa komponen keamanan, yaitu RFID untuk autentikasi kartu akses, sensor getaran untuk mendeteksi gangguan fisik pada brankas, sensor pintu berbasis magnetic reed switch untuk memantau status pintu, serta servo motor untuk mekanisme buka tutup pintu secara otomatis. MCU 2 mempublikasikan data status akses RFID, kondisi pintu, getaran, dan peringatan keamanan ke MQTT broker.



### 3. MCU 3 - Monitoring

MCU 3 berfungsi sebagai subscriber yang menerima semua pesan alert dari broker. Perangkat ini dilengkapi LED merah dan hijau serta buzzer sebagai indikator fisik langsung di lapangan. Ketika terjadi alert, MCU 3 akan memberikan respons visual dan audio sesuai jenis peringatan yang diterima, sehingga operator di lokasi dapat mengetahui kondisi darurat.

### 4. MQTT Broker (10.33.109.55)

MQTT broker berjalan pada VPS kampus dengan alamat IP 10.33.109.55 menggunakan software Mosquitto. Broker ini menjadi pusat komunikasi yang menerima semua data yang dipublikasikan oleh MCU 1 dan MCU 2, kemudian mendistribusikannya ke semua subscriber yang terdaftar, termasuk MCU 3 dan dashboard monitoring. Penggunaan protokol MQTT dipilih karena ringan, efisien dalam penggunaan bandwidth, dan cocok untuk komunikasi antar perangkat IoT.

Pada implementasi proyek ini, MQTT Broker ditempatkan pada VPS kampus agar dapat diakses secara terpusat oleh seluruh perangkat IoT melalui jaringan internet. Sementara itu, Node-RED, Grafana, dan Apache Hadoop HDFS dijalankan pada VM Ubuntu lokal yang digunakan sebagai lingkungan pengembangan dan pengujian sistem. Arsitektur ini memungkinkan komunikasi data dilakukan secara terpusat melalui VPS, sedangkan proses pemrosesan, visualisasi, dan penyimpanan data dapat dikelola secara fleksibel pada server lokal.

### 5. Node-RED (Processing Layer)

Node-RED berfungsi sebagai middleware yang menerima data dari MQTT broker untuk dilakukan pemrosesan lebih lanjut. Pada sistem ini, Node-RED digunakan untuk melakukan parsing data sensor, pengelompokan data berdasarkan topik MQTT, pengelolaan status alert, serta meneruskan data menuju dashboard monitoring dan sistem penyimpanan HDFS. Selain itu, Node-RED juga digunakan untuk mengatur mekanisme backup dan integrasi antar komponen sistem sehingga alur komunikasi dapat berjalan secara otomatis dan real-time.

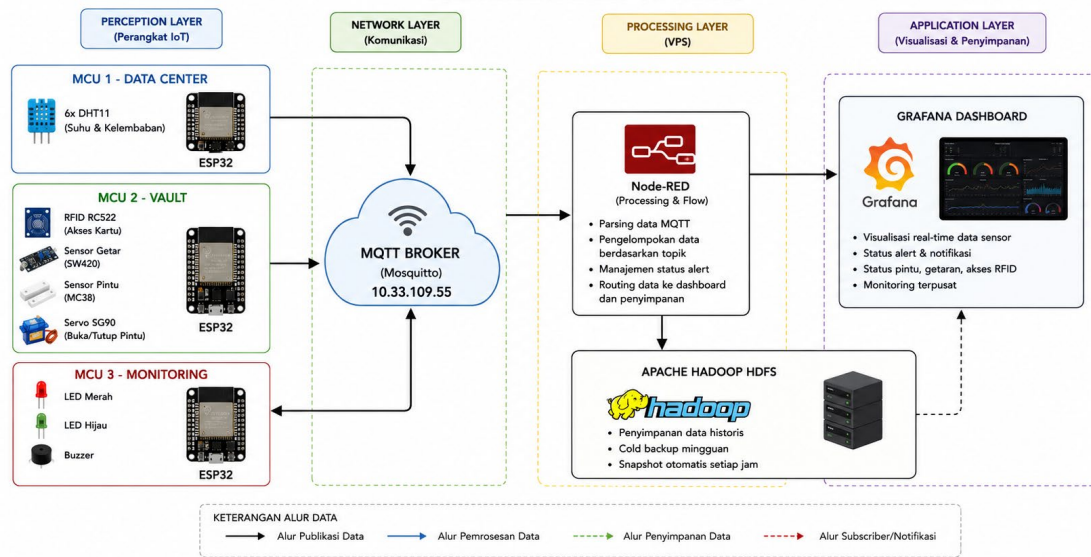
### 6. Dashboard - Visualisasi Real-Time

Dashboard utama dibangun menggunakan Grafana yang berjalan pada VM Ubuntu lokal. Data sensor yang diterima melalui MQTT broker terlebih dahulu diproses oleh Node-RED sebelum ditampilkan pada dashboard. Grafana digunakan untuk menampilkan visualisasi real-time berupa suhu dan kelembaban data center, status getaran, status pintu, akses RFID, serta kondisi alert sistem. Dashboard ini membantu operator dalam memantau kondisi infrastruktur perbankan secara terpusat dan real-time.

## 7. Apache Hadoop HDFS (Data Storage Layer)

Apache Hadoop Distributed File System (HDFS) digunakan sebagai media penyimpanan data historis yang berasal dari seluruh perangkat IoT. Data yang telah diterima dan diproses oleh Node-RED disimpan ke dalam HDFS untuk keperluan arsip, backup, dan analisis data jangka panjang. Pada sistem ini diterapkan mekanisme snapshot otomatis setiap satu jam dan cold backup mingguan guna meningkatkan keandalan penyimpanan data monitoring.

### 3.1.2 Arsitektur Sistem



Gambar 2. Arsitektur Sistem

Arsitektur sistem Early Warning Infrastruktur Bank dirancang menggunakan konsep Internet of Things (IoT) yang terdiri atas empat lapisan utama, yaitu *Perception Layer*, *Network Layer*, *Processing Layer*, dan *Application Layer*. Arsitektur ini memungkinkan proses pengambilan data, komunikasi, pemrosesan, penyimpanan, serta visualisasi data berlangsung secara terintegrasi dan real-time.

#### 1. Perception Layer (Perangkat IoT)

Perception Layer merupakan lapisan yang bertugas melakukan akuisisi data dari lingkungan fisik. Pada sistem ini terdapat tiga unit mikrokontroler ESP32 yang memiliki fungsi berbeda. MCU 1 berfungsi sebagai unit monitoring data center yang dilengkapi enam sensor DHT11 untuk mengukur suhu dan kelembaban. Data yang diperoleh digunakan untuk memantau kondisi lingkungan ruang server dan menghasilkan peringatan apabila suhu melebihi 30°C atau kelembaban melebihi 50%.

MCU 2 berfungsi sebagai unit keamanan vault yang terintegrasi dengan RFID RC522 untuk autentikasi akses, sensor getaran SW420 untuk mendeteksi gangguan fisik, sensor pintu MC38 untuk memantau kondisi pintu, serta motor servo SG90 sebagai mekanisme buka-tutup pintu otomatis.

MCU 3 berfungsi sebagai perangkat monitoring lokal yang menerima pesan peringatan dari sistem. Perangkat ini dilengkapi LED merah, LED hijau, dan buzzer sebagai indikator visual dan audio ketika terjadi kondisi abnormal.

## **2. Network Layer (Komunikasi Data)**

Network Layer berfungsi sebagai media komunikasi antar perangkat dalam sistem. Seluruh perangkat ESP32 terhubung ke jaringan Wi-Fi dan berkomunikasi menggunakan protokol MQTT dengan model publish-subscribe. MQTT Broker yang menggunakan Mosquitto dan berjalan pada VPS kampus dengan alamat IP 10.33.109.55 bertindak sebagai pusat komunikasi sistem. Broker menerima seluruh data yang dipublikasikan oleh MCU 1 dan MCU 2, kemudian mendistribusikannya kepada subscriber yang terdaftar, termasuk Node-RED dan MCU 3.

## **3. Processing Layer (Pemrosesan Data)**

Processing Layer dijalankan pada VM Ubuntu lokal menggunakan Node-RED sebagai middleware utama. Node-RED bertugas menerima data dari MQTT Broker, melakukan parsing data, mengelompokkan data berdasarkan topik, mengelola status alert, serta meneruskan data ke dashboard monitoring dan sistem penyimpanan. Selain itu, Node-RED juga digunakan untuk mengintegrasikan berbagai komponen sistem sehingga proses monitoring dan notifikasi dapat berjalan secara otomatis dan real-time.

## **4. Application Layer (Visualisasi dan Penyimpanan Data)**

Application Layer terdiri atas Grafana dan Apache Hadoop HDFS. Grafana digunakan sebagai dashboard utama untuk menampilkan data monitoring secara real-time, meliputi suhu dan kelembaban data center, status getaran, kondisi pintu, akses RFID, serta status peringatan sistem.

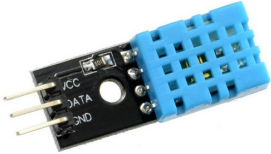
Sementara itu, Apache Hadoop Distributed File System (HDFS) digunakan sebagai media penyimpanan data historis yang berasal dari seluruh perangkat IoT. Data yang diterima melalui Node-RED disimpan ke dalam HDFS untuk keperluan arsip, analisis data, dan backup. Sistem penyimpanan ini mendukung mekanisme snapshot otomatis setiap satu jam dan cold backup mingguan guna meningkatkan keandalan penyimpanan data.

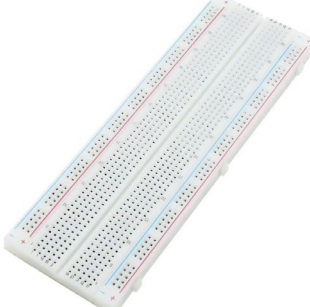
Secara keseluruhan, aliran data pada sistem dimulai dari sensor yang terhubung ke ESP32, kemudian dikirim ke MQTT Broker melalui jaringan Wi-Fi. Data selanjutnya diproses oleh Node-RED, disimpan ke HDFS, dan divisualisasikan melalui Grafana. Ketika kondisi abnormal terdeteksi, sistem akan mengirimkan notifikasi kepada MCU 3 sehingga LED dan buzzer dapat memberikan peringatan secara langsung kepada operator.

### 3.2 Alat dan Bahan

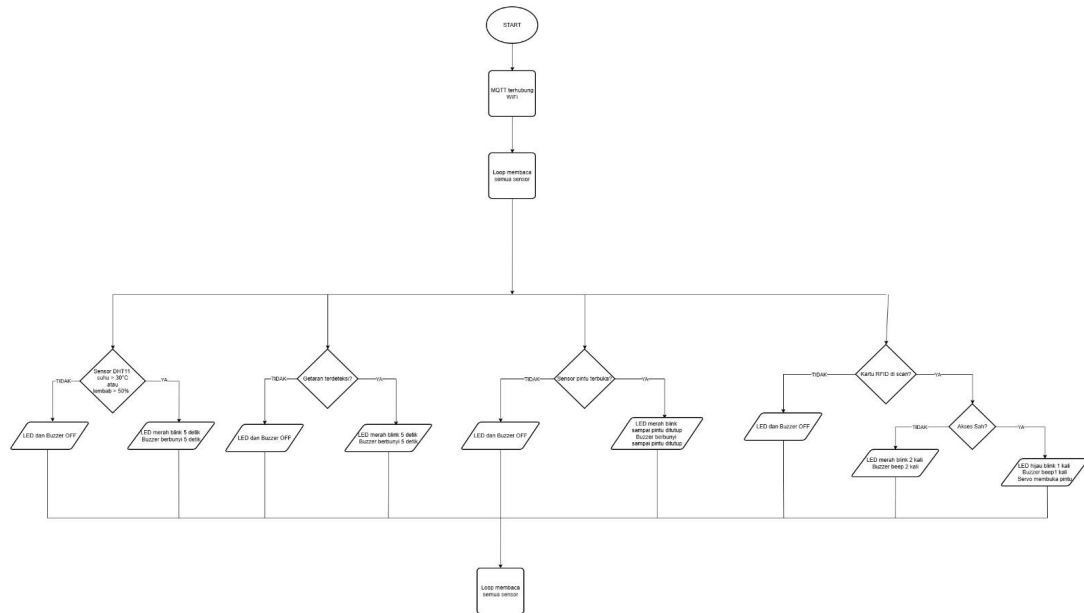
Alat yang digunakan pada project kali ini adalah:

Table 1. Alat dan Bahan

| No | Nama Alat dan Bahan        | Spesifikasi               | Jumlah | Keterangan                                                                            |
|----|----------------------------|---------------------------|--------|---------------------------------------------------------------------------------------|
| 1. | Sensor Suhu dan kelembaban | DHT11                     | 6      |    |
| 2. | Sensor Getar               | SW420                     | 1      |   |
| 3. | Sensor Pintu               | MC38                      | 1      |  |
| 4. | LED                        | 5mm (3 merah dan 1 hijau) | 4      |  |
| 5. | Buzzer                     | 2.5V                      | 2      |  |

|     |                 |            |    |                                                                                       |
|-----|-----------------|------------|----|---------------------------------------------------------------------------------------|
| 6.  | Mikrokontroler  | ESP32      | 3  |    |
| 7.  | Kabel Jumper    | MF, MM, FF | 50 |    |
| 8.  | Breadboard      | MB102      | 3  |   |
| 9.  | Sensor RFID Kit | RC522      | 1  |  |
| 10. | Motor Servo     | SG90       | 1  |  |

### 3.3 Diagram Alur Kerja (Flowchart)



Gambar 3. Flowchart Sistem

Diagram alur kerja dimulai dengan menghubungkan MQTT ke jaringan Wi-Fi UGM-Hotspot. Di dalam loop program, seluruh sensor dibaca untuk kemudian dikirimkan ke MQTT. Sensor-sensor tersebut meliputi:

1. Sensor DHT11: Digunakan enam buah sensor DHT11 yang ditempatkan di area Data Center untuk memantau suhu dan kelembaban. Logika yang diterapkan adalah jika suhu terdeteksi melebihi 30°C atau kelembaban melebihi 50%, maka lampu LED dan buzzer akan aktif selama 5 detik.
2. Sensor Getaran: Digunakan untuk mendeteksi getaran pada brankas. Jika getaran terdeteksi, maka LED dan buzzer akan aktif selama 5 detik.
3. Sensor Pintu: Dipasang pada pintu ruang brankas untuk memantau kondisi terbuka atau tertutup. Jika pintu terbuka, LED akan berkedip dan buzzer akan berbunyi hingga pintu tertutup kembali.
4. RFID: Dikonfigurasi untuk dua kondisi akses. Kondisi pertama, jika akses sah, LED hijau akan berkedip satu kali, buzzer berbunyi satu kali, dan servo membuka pintu. Kondisi kedua, jika akses tidak sah, LED merah akan berkedip selama 2 detik dan buzzer akan berbunyi selama 2 detik.

### 3.4 Desain Perangkat Keras (Hardware)

#### 3.4.1 I/O Mapping

Berdasarkan pemetaan I/O yang telah dilakukan, setiap mikrokontroler ESP32 memiliki fungsi yang berbeda sesuai dengan kebutuhan sistem. MCU 1 digunakan untuk monitoring lingkungan data center dengan menghubungkan enam sensor DHT11 yang berfungsi membaca suhu dan kelembaban pada beberapa titik pengukuran. MCU 2 berperan sebagai sistem keamanan vault yang terintegrasi dengan RFID RC522, sensor getaran SW420, sensor pintu MC38, dan motor servo SG90 untuk mendukung autentikasi akses serta deteksi gangguan fisik. Sementara itu, MCU 3 digunakan sebagai unit monitoring lokal yang mengendalikan LED dan buzzer sebagai indikator visual maupun audio ketika sistem mendeteksi kondisi abnormal. Pemetaan pin yang tepat diperlukan untuk memastikan seluruh sensor dan aktuator dapat berkomunikasi dengan ESP32 secara optimal serta mendukung proses monitoring dan peringatan dini secara real-time.

#### A. MCUI 1: Data Center

*Table 2. Pin MCU + Data Center*

| DHT 11     | PIN  |
|------------|------|
| DHT 11 # 1 | G 16 |
| DHT 11 # 2 | G 17 |
| DHT 11 # 3 | G 18 |
| DHT 11 # 4 | G 25 |
| DHT 11 # 5 | G 26 |
| DHT 11 # 6 | G 27 |

## B. MCU 2: Vault

Table 3. Pin MCU + Vault

| Sensor    | Pin   |
|-----------|-------|
| Getar VCC | 5 V   |
| Getar DO  | 34    |
| Pintu     | 35    |
| Servo VCC | 5 V   |
| Servo DO  | 13    |
| SDA/SS    | 4     |
| SCK       | 18    |
| MOSI      | 23    |
| MISO      | 19    |
| RST       | 22    |
| RFID VCC  | 3.3 V |

## C. MCU 3: Monitoring

Table 4. Pin MCU + Aktuator

| Aktuator          | Pin |
|-------------------|-----|
| LED Merah DC      | 16  |
| Buzer DC          | 17  |
| LED Merah Getaran | 21  |
| LED Hijau Vault   | 32  |
| LED Merah Vault   | 33  |
| Buzer Vault       | 25  |



### 3.4.2 Pemrograman Hardware

Pemrograman perangkat keras pada sistem ini dilakukan menggunakan Arduino IDE dengan bahasa pemrograman C++. Setiap mikrokontroler ESP32 diprogram sesuai dengan fungsi yang dimiliki dalam sistem. MCU 1 digunakan untuk membaca data suhu dan kelembaban dari enam sensor DHT11 yang terpasang pada area data center, kemudian mengolah dan mengirimkan data tersebut ke MQTT Broker melalui jaringan Wi-Fi. MCU 2 diprogram untuk mengelola sistem keamanan vault yang mencakup autentikasi kartu RFID, deteksi getaran, pemantauan status pintu, serta pengendalian motor servo sebagai mekanisme buka-tutup pintu secara otomatis.

Sementara itu, MCU 3 diprogram sebagai perangkat monitoring lokal yang berfungsi menerima pesan dari MQTT Broker menggunakan metode subscribe pada topik tertentu. Berdasarkan data yang diterima, MCU 3 akan mengendalikan LED dan buzzer sebagai indikator visual dan audio ketika terjadi kondisi abnormal pada sistem. Seluruh program dirancang menggunakan pendekatan komunikasi MQTT berbasis publish-subscribe sehingga proses pertukaran data antar perangkat dapat berlangsung secara real-time, ringan, dan efisien untuk mendukung implementasi sistem early warning infrastruktur perbankan.

## 3.5 Desain Perangkat Lunak (Software)

### 3.5.1 Algoritma utama

Sistem monitoring infrastruktur bank ini terdiri dari tiga unit mikrokontroler yang terintegrasi melalui protokol MQTT. MCU 1 bertugas memantau suhu dan kelembaban data center dari enam sensor DHT11, menghitung nilai rata-rata, serta mengirimkan alert jika melebihi batas normal. MCU 2 mengelola keamanan vault dengan integrasi RFID, sensor getaran, sensor pintu, dan servo pengunci. MCU 3 berfungsi sebagai unit output, mengaktifkan LED dan buzzer sesuai dengan notifikasi yang diterima dari MQTT broker. Seluruh data dikirim ke VPS, diproses oleh Node-RED, disimpan ke Hadoop HDFS, serta ditampilkan pada dashboard monitoring real-time. Berikut adalah algoritma utama dari ketiga mikrokontroler tersebut:

#### A. MCU 1: Algoritma Monitoring Suhu dan Kelembaban:

1. Inisialisasi WiFi, MQTT, dan 6 sensor DHT11.
2. Loop:
  - a. Baca nilai suhu dan kelembaban dari 6 sensor.
  - b. Hitung rata-rata suhu dan kelembaban.
  - c. Cek kondisi alert:
    - Jika suhu  $> 30^{\circ}\text{C}$  ATAU kelembaban  $> 50\%$ , maka publish "1" ke bank/dc/alert.
    - Jika tidak, maka publish "0" ke bank/dc/alert.
  - d. Publish rata-rata ke bank/dc/temperature dan bank/dc/humidity.

- e. Publish nilai per sensor ke bank/dc/sensor[n]/suhu dan bank/dc/sensor[n]/hum.
  - f. Kirim heartbeat setiap 30 detik ke bank/dc/heartbeat.
  - g. Delay 3 detik, lalu mengulang dari langkah a.
- B. MCU 2: Algoritma Keamanan Vault:
1. Inisialisasi WiFi, MQTT, RFID, sensor getaran, sensor pintu, servo.
  2. Servo default posisi tertutup (0°)
  3. Loop:
    - a. [RFID] Cek kartu baru:
      - Jika kartu valid, maka publish “granted”, buka servo (90°), set aksesAman = true.
      - Jika kartu tidak valid, maka publish “denied” + “akses\_illegal”.
    - b. [Akses] Jika aksesAman dan sudah 5 detik:
      - Tutup servo (0°), publish “akses\_berakhir”.
    - c. [Getaran] Baca sensor getaran:
      - Jika terdeteksi, maka publish “1”, aktifkan timer 5 detik.
      - Jika 5 detik setelah getaran terakhir selesai, maka publish “0”.
    - d. [Pintu] Deteksi perubahan status pintu:
      - Jika terbuka + aksesAman, maka publish “open\_sah”.
      - Jika terbuka + tidak ada akses, maka publish “open\_tanpa\_akses” + alert.
      - Jika tertutup, maka publish “closed”.
    - e. [Kombinasi] Jika kartu illegal + pintu terbuka:
      - Publish “akses\_illegal\_pintu\_terbuka”.
    - f. Delay 10ms, lalu mengulangi dari langkah a.
- C. MCU 3: Algoritma Monitoring Fisik:
1. Inisialisasi WiFi, MQTT, LED, Buzzer.
  2. Subscribe ke topic: bank/dc/alert, bank/vault/alert, bank/vault/vibration, bank/vault/access, bank/vault/door.
  3. Loop:
    - a. Terima pesan MQTT via callback:
      - bank/dc/alert = “1”, maka LED merah DC + buzzer DC berkedip 5 detik.
      - bank/vault/vibration = “1”, maka LED merah vault + buzzer vault berkedip.
      - bank/vault/access = “granted”, maka LED hijau vault + buzzer 1x.
      - bank/vault/access = “denied”, maka LED merah vault + buzzer 2x.
      - bank/vault/door = “open\_tanpa\_akses”, maka LED merah vault berkedip terus.
      - bank/vault/door = “open\_sah”, maka LED hijau vault menyala.

- bank/vault/door = “closed”, maka semua LED vault mati.
- b. Update timer dan status LED secara non-blocking.
- c. Delay selama 10ms, lalu mengulangi dari langkah a.

### 3.5.2 Bahasa pemrograman/platform

Table 5. Bahasa Pemrograman

| Komponen                   | Platform / Bahasa     | Keterangan                                    |
|----------------------------|-----------------------|-----------------------------------------------|
| MCU 1, 2, 3                | Arduino IDE / C++     | Pemrograman mikrokontroler ESP32              |
| MQTT Broker                | Mosquitto / Linux     | Berjalan di VPS kampus (10.33.109.55)         |
| Middleware & Dashboard IoT | Node-RED / JavaScript | Flow-based programming, berjalan di VM Ubuntu |
| Visualisasi Utama          | Grafana               | Dashboard monitoring real-time                |
| Penyimpanan Big Data       | Apache Hadoop (HDFS)  | VM Ubuntu lokal, Hadoop 3.3.6                 |
| Sistem Operasi lokal       | Ubuntu 22.04 LTS      | VM VMware Workstation                         |

### 3.5.3 Interface aplikasi

Sistem ini memiliki dua layer interface yang dapat diakses oleh operator:

1. Node-RED Flow dan Dashboard (<http://192.168.31.129:1880/ui>)

Node-RED berperan ganda dalam sistem ini, yaitu sebagai middleware sekaligus antarmuka manajemen alur data. Melalui editor Node-RED (/1880), operator dapat melihat alur data secara visual, dan dapat memantau jadwal backup yang telah dikonfigurasi dalam flow, yaitu cold backup otomatis setiap hari Jumat ke folder /data-backup di HDFS dan snapshot otomatis setiap satu jam ke folder /data-realtime dan /data-backup.

Melalui Node-RED Dashboard (/ui), digunakan pada fase pengujian awal untuk memverifikasi apakah data dari sensor berhasil diterima melalui MQTT sebelum dipindahkan ke Grafana sebagai dashboard utama. Tampilan meliputi gauge suhu dan kelembaban per sensor, status getaran, status pintu, dan akses RFID dalam bentuk teks real-time.

2. Grafana Dashboard (<http://192.168.31.129:3000>)

Dashboard utama yang digunakan untuk monitoring operasional dengan nama “Bank Early Warning”. Grafana menyediakan fitur bawaan auto-refresh dan pemilihan rentang waktu data yang dapat dipilih langsung oleh operator melalui antarmuka dashboard seperti “Last 5 minute” yang terlihat pada tampilan sistem. Dashboard Grafana menampilkan:

- a) Gauge rata-rata suhu dan kelembaban seluruh data center.
- b) Gauge suhu dan kelembaban individual per rak server (Rak 1-6).
- c) Panel status alert data center dengan value mapping teks.
- d) Panel sensor getaran, status pintu, dan akses RFID vault dengan value mapping.

- e) Sistem warna tiga tingkat; hijau (normal), kuning (mendekati batas), merah (kritis).

### 3.6 Desain Dashboard (Software)

Dashboard sistem ini dilakukan dalam dua tahap. Pada tahap awal, digunakan Node-RED Dashboard sebagai media pengujian untuk memverifikasi apakah data sensor berhasil dikirim melalui protokol MQTT dan diterima dengan benar oleh sistem. Node-RED dashboard dipilih karena terintegrasi langsung dengan flow Node-RED sehingga memungkinkan pengujian cepat tanpa konfigurasi tambahan. Setelah seluruh data dipastikan berjalan, pengembangan dilanjutkan ke tahap kedua menggunakan Grafana sebagai platform dashboard utama.

Dashboard utama dibangun menggunakan Grafana dengan nama “Bank Early Warning” yang dapat diakses melalui browser di alamat <http://192.168.31.129:3000>. Data source yang digunakan adalah plugin MQTT Grafana yang subscribe langsung ke broker Mosquitto di VPS kampus (10.33.109.55), sehingga seluruh data ditampilkan secara real-time.

Dashboard dibagi menjadi beberapa panel yang mencakup seluruh komponen sensor sistem:

1. Panel Rata-rata Suhu dan Kelembaban

Dua panel *gauge* rata-rata suhu dan kelembaban di bagian tengah menampilkan nilai rata-rata suhu dan kelembaban dari keenam sensor DHT11 yang terpasang di rak server. Pada dashboard yang ditampilkan, digunakan sistem warna tiga tingkat, untuk suhu maupun kelembaban; hijau untuk kondisi suhu di bawah 25°C, kuning untuk kondisi suhu di antara 25 - 30°C, dan merah untuk suhu yang melampaui 30°C. Sementara kelembaban; hijau untuk kondisi kelembaban di bawah 48%, kuning untuk kelembaban di antara 48-52%, dan warna merah untuk kelembaban di atas 52%.

2. Panel Status Data Center

Panel Status Data Center menampilkan seluruh status alert keseluruhan sistem data center. Sistem data center meliputi rata-rata suhu dan rata-rata kelembaban. Jadi, jika salah satu panel rata-rata melebihi *threshold*, maka panel status akan menampilkan teks berwarna merah “Suhu/Kelembaban Melebihi Batas!”. Teks tersebut dapat tertampil karena sebelumnya nilai mapping telah dikonfigurasi sehingga nilai 1 dari MCU 1 diterjemahkan menjadi teks peringatan yang informatif, bukan sekedar angka.

3. Panel Suhu dan Kelembaban Per Sensor (1-6)

Dua baris panel *gauge* di bagian bawah menampilkan pembacaan individual dari masing-masing sensor suhu dan kelembaban). Pada tiap panel tersebut juga sudah dikonfigurasi sistem warna tiga tingkat dengan *threshold* yang sama. Panel ini memungkinkan operator untuk mengidentifikasi rak mana yang mengalami kondisi kritis secara spesifik.

4. Panel Sensor Getaran

Panel Sensor Getaran subscribe ke topic bank/vault/vibration yang dipublikasikan oleh MCU 2. Sensor getaran dipasang pada brankas yang akan menghasilkan sinyal HIGH

saat getaran terdeteksi. MCU 2 mempublikasikan nilai “1” ketika getaran terdeteksi dan nilai “0” saat tidak ada getaran selama 5 detik. Di sisi Grafana, nilai mentah tersebut diterjemahkan menggunakan fitur Value Mapping dengan konfigurasi;

- a) Value : 1 menjadi GETARAN TERDETEKSI.
- b) Value: 0 menjadi AMAN.

Dengan konfigurasi ini, panel menampilkan status yang mudah dibaca oleh operator tanpa perlu menafsirkan angka. Ketika tidak ada aktivitas di area vault, panel menampilkan "AMAN", dan berubah menjadi "GETARAN TERDETEKSI" secara real-time ketika sensor aktif.

## 5. Panel Status Pintu

Panel Status Pintu subscribe ke topic bank/vault/door. Sensor pintu yang digunakan adalah *magnetic reed switch* yang bersifat normally closed, dengan mekanisme terhubung saat magnet menempel (pintu tertutup) dan terputus saat pintu menjauh (pintu terbuka).

MCU 2 mempublikasikan tiga kemungkinan nilai ke topic ini:

- a) “open\_sah” : pintu terbuka setelah kartu RFID valid berhasil diverifikasi.
- b) “open\_tanpa\_akses” : pintu terbuka tanpa otorisasi RFID.
- c) “closed” : pintu kembali tertutup.

Value mappings yang dikonfigurasi di Grafana:

- a) Value: open\_sah menjadi PINTU TERBUKA - AKSES SAH.
- b) Value: open\_tanpa\_akses menjadi PINTU TERBUKA - TANPA OTORISASI.
- c) Value: closed menjadi PINTU TERTUTUP.

Panel ini penting karena membedakan antara pembukaan pintu yang sah dan yang mencurigakan, sehingga operator dapat mengambil tindakan yang tepat.

## 6. Panel Akses RFID

Panel “Akses RFID” subscribe ke topic bank/vault/access. Modul RFID RC522 pada MCU 2 membaca UID kartu yang didekatkan dan mencocokkannya dengan daftar kartu yang telah didaftarkan dalam kode program.

MCU 2 mempublikasikan tiga kemungkinan nilai:

- a) “Granted” : kartu yang dikenali sebagai kartu valid.
- b) “denied” : kartu tidak dikenali atau tidak terdaftar.
- c) “akses\_berakhir”: masa akses 5 detik telah habis dan servo menutup pintu kembali.

Value mappings yang dikonfigurasi di Grafana:

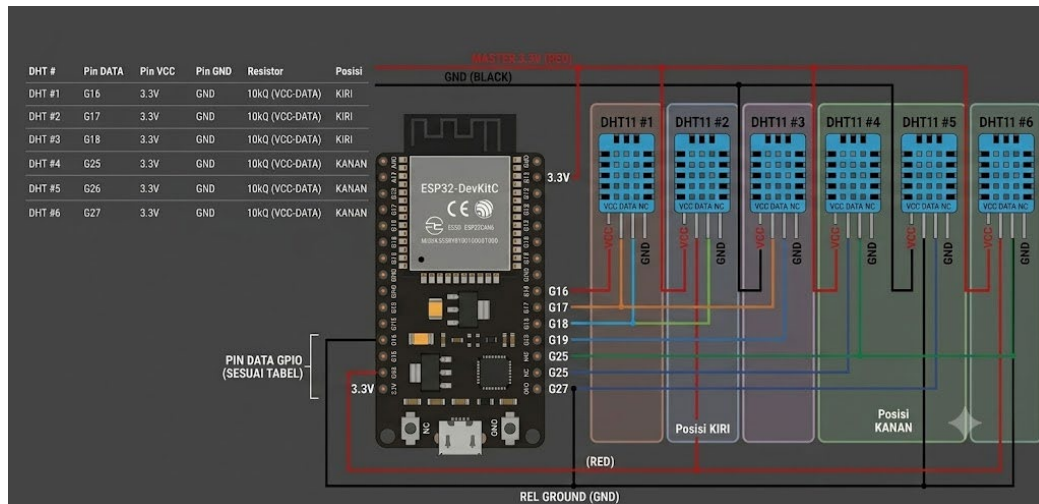
- a) Value: granted menjadi AKSES DITERIMA.
- b) Value: denied menjadi AKSES DITOLAK.
- c) Value: akses\_berakhir menjadi MASA AKSES BERAKHIR.

Panel ini memungkinkan monitoring akses masuk ke area vault secara real-time, sehingga setiap percobaan akses baik yang berhasil maupun yang ditolak dapat terpantau langsung dari dashboard.

## D. IMPLEMENTASI DAN PENGUJIAN

### 4.1 Implementasi Rangkaian dan Program

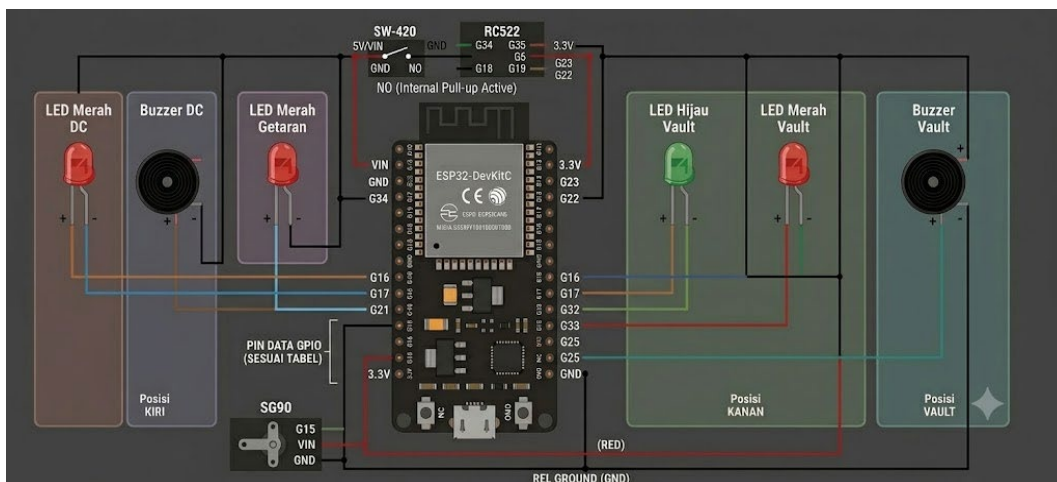
#### 4.1.1 Data Center



Gambar 4. Peta Wiring MCU + DHT11

Pada rangkaian ini, terdapat satu modul ESP32 yang terhubung ke enam sensor DHT11. Keenam sensor dibagi menjadi dua kelompok, terletak tiga di sisi kiri dan tiga di sisi kanan. Setiap sensor DHT11 menggunakan tiga jalur koneksi, yaitu VCC (3.3 V), GND, dan DATA. Pin DATA dari masing - masing sensor dihubungkan ke pin GPIO digital ESP32 secara berurutan, seperti yang sudah tertera di tabel. Pemilihan pin - pin ini didasarkan pada ketersediaan GPIO digital ESP32 yang mendukung komunikasi *single wire* DHT11.

#### 4.1.2 Aktuator

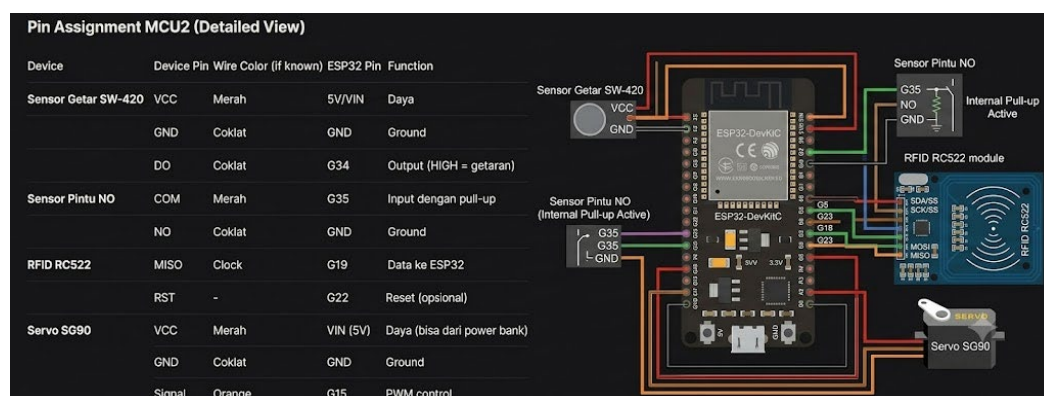


Gambar 5. Peta Wiring MCU + Aktuator

Pada rangkaian ini, terdapat satu modul ESP32 yang terhubung ke enam aktuator yang terdiri dari dua *buzzer*, satu LED hijau dan tiga LED merah. Aktuator ini dibagi menjadi dua sisi, yaitu sisi kiri dan sisi kanan. Sisi kiri terdapat tiga aktuator, yaitu LED merah DC yang terhubung ke pin G16, *buzzer* DC yang terhubung ke G17, dan LED merah getaran yang terhubung ke G21. Ketiga aktuator ini berfungsi sebagai indikator yang menunjukkan kondisi data center, dimana LED merah DC akan menyala apabila suhu tidak normal, *buzzer* berbunyi saat suhu tidak normal, dan LED merah getaran akan berbunyi saat terdapat getaran yang terdeteksi.

Pada sisi kanan, terdapat tiga aktuator, yaitu LED hijau *vault* pada pin G32, LED merah *vault* pada pin G33, dan *buzzer* pada pin G25. ketiga aktuator ini berfungsi sebagai indikator status akses ruang *vault*, LED hijau *vault* sebagai indikator bahwa akses RFID valid dan pintu dibuka oleh servo, LED merah *vault* sebagai indikator dari akses RFID tidak valid atau pintu terbuka tanpa otorisasi. *Buzzer* sebagai indikator dari RFID akses valid yang akan berbunyi *beep* sekali, berbunyi dua kali apabila akses RFID tidak valid maupun saat pintu terbuka tanpa otorisasi yang akan berbunyi terus menerus sampai pintu kembali ditutup.

### 4.1.3 Vault



Gambar 6. Peta Wiring MCU + Vault

Pada rangkaian ini, terdapat satu modul ESP32 yang terhubung ke tiga sensor yang berbeda, yaitu sensor getar, sensor pintu, dan RFID. serta aktuator servo. Sensor getaran terhubung ke tiga pin, yaitu VCC (5 V), GND, dan G34. sensor ini akan menghasilkan hasil logika *high* apabila mendeteksi gerakan, sehingga ESP32 dapat membaca kondisi ini sebagai indikasi adanya gangguan fisik pada rak server. Sensor pintu NC terhubung ke pin GND dan G35. pada kondisi normal, pintu akan tertutup, dan ketika terbuka ESP32 akan membaca logika *high*.

Modul RFID menggunakan komunikasi SPI (Serial Peripheral Interface) yang membutuhkan beberapa jalur khusus: SDA/SS, SCK, MOSI, dan MISO yang terhubung ke pin G5, G18, G23, dan G19 secara berurutan, serta pin RST yang terhubung ke G22 untuk fungsi reset modul (bersifat opsional). Komunikasi SPI dipilih karena mampu memberikan kecepatan transfer data yang lebih tinggi dibandingkan komunikasi serial biasa, sehingga proses pembacaan UID kartu dapat dilakukan dengan cepat dan akurat. Motor servo

menggunakan tiga jalur, yaitu VCC (5 V), GND, dan pin G13 yang berfungsi mengirimkan sinyal PWM (*Pulse Width Modulation*) untuk mengatur posisi sudut putaran servo. Pada sistem ini, servo berfungsi sebagai aktuator mekanis penutup/pembuka pintu, yang akan bergerak ke posisi terbuka ketika RFID membaca kartu dengan UID yang valid, dan kembali ke posisi terkunci pada kondisi sebaliknya.

## 4.2 Konfigurasi Platform IoT

Platform IoT pada sistem ini terdiri dari tiga komponen utama yang saling terintegrasi, yaitu MQTT Broker sebagai penghubung komunikasi, Node-Red sebagai *middleware* pemrosesan data, dan Apache Hadoop sebagai media penyimpanan data skala besar. Ketiga komponen ini dikonfigurasi pada dua lingkungan berbeda untuk mengoptimalkan penggunaan sumber daya.

### 4.2.1 Konfigurasi MQTT Broker

MQTT Broker menggunakan mosquitto yang di unduh pada VPS dengan alamat IP 10.33.109.55. Broker ini berfungsi sebagai titik penerima data dari seluruh perangkat ESP32 melalui protokol *publish-subscribe*. Konfigurasi mosquitto berisi **listener 1883 0.0.0.0** yang memungkinkan broker menerima koneksi dari seluruh antarmuka jaringan, sehingga ESP32 yang terhubung melalui jaringan WiFi dapat mengakses broker. Parameter **allow\_anonymous true** mengizinkan koneksi tanpa autentikasi, digunakan untuk mempermudah proses pengembangan dan pengujian sistem. Topik MQTT yang dikonfigurasi adalah vibradoor/getaran, vibradoor/pintu, vibradoor/rfid, vibradoor/alert, vibradoor/suhu.

### 4.2.2 Konfigurasi Node-RED

Node-RED diunduh pada VM lokal dan berperan sebagai *middleware* yang menjembatani data dari MQTT broker ke sistem penyimpanan dan *dashboard* monitoring. Konfigurasi koneksi MQTT pada Node-RED diatur melalui node mqtt-broker. Setiap topik MQTT *disubscribe* melalui node mqtt in terpisah, kemudian diteruskan ke node function untuk diproses. Pemrosesan data pada Node-RED meliputi parsing JSON yang mengubah payload MQTT menjadi objek JavaScript yang dapat diolah. Selanjutnya penyimpanan state untuk menyimpan kondisi terkini seluruh sensor dalam satu objek, selanjutnya debounce dan filtering yang mencegah pembacaan status pintu yang berubah - ubah dalam waktu singkat, selanjutnya auto-reset yang mengembalikan status getaran ke kondisi normal secara otomatis setelah lima detik tanpa sinyal getaran baru, dan terakhir penghitungan agregat yang menghitung rata-rata suhu dari enam sensor untuk ditampilkan pada panel “rata - rata data center”.

### 4.2.3 Konfigurasi Penyimpanan Hadoop (HDFS)

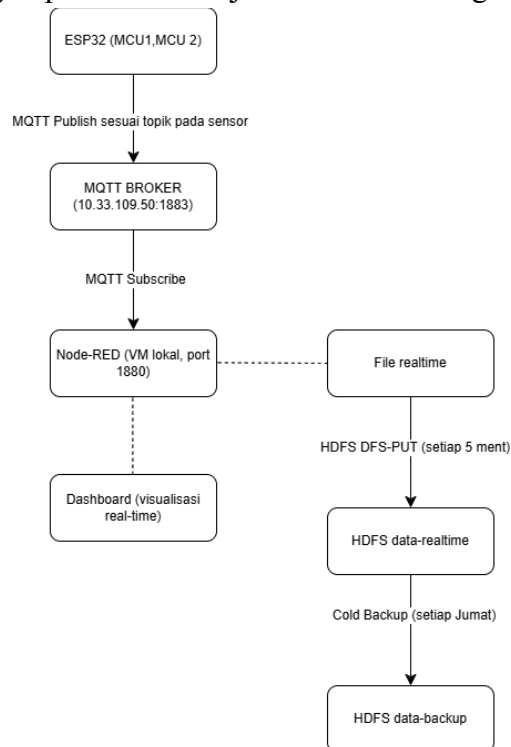
Apache Hadoop diunduh pada VM, sama seperti Node-RED dalam mode *pseudo-distributed*. Dua direktori utama yang dibuat adalah *data-realtime* yang menyimpan data



sensor terkini yang diperbarui secara berkala dan *data-backup* menyimpan salinan data historis untuk keperluan *backup* mingguan. Node-RED terhubung ke HDFS melalui dua mekanisme, yaitu penyimpanan sementara dan *upload* berkala. Selain itu, sistem juga dikonfigurasi untuk melakukan *cold backup* setiap hari jum'at pukul 00:00 dengan menyalin data dari *data-realtime* ke *data-backup* menggunakan format json, serta membuat *snapshot* HDFS setiap satu jam mekanisme pemulihan data.

#### 4.2.4 Diagram Alur Konfigurasi Platform

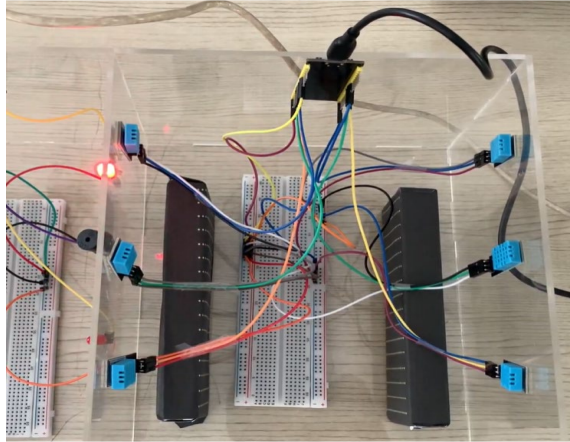
Diagram alur konfigurasi platform menunjukkan proses integrasi antar komponen pada sistem IoT yang dikembangkan. Data yang diperoleh dari perangkat ESP32 dikirimkan ke MQTT Broker melalui jaringan Wi-Fi menggunakan protokol MQTT. Selanjutnya, data diterima dan diproses oleh Node-RED sebelum diteruskan ke Grafana untuk visualisasi monitoring secara real-time serta disimpan ke Apache Hadoop HDFS sebagai media penyimpanan data historis. Alur ini memungkinkan proses monitoring, pengolahan, dan penyimpanan data berjalan secara terintegrasi.



## 4.3 Pengujian Sistem

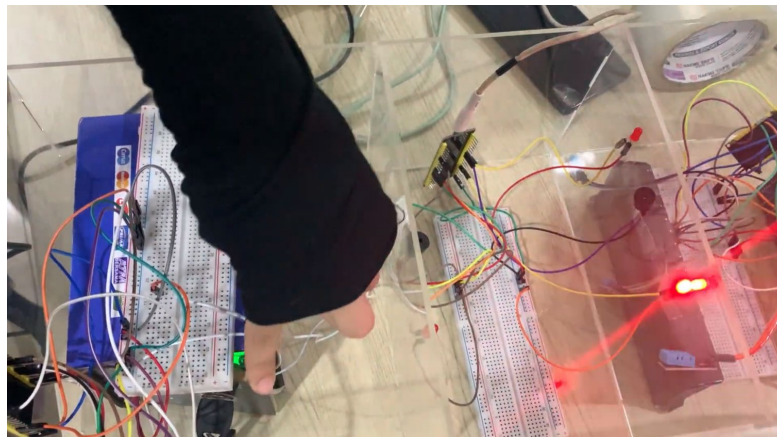
### 4.3.1 Pengujian Sensor

#### a. Pengujian Sensor DHT11



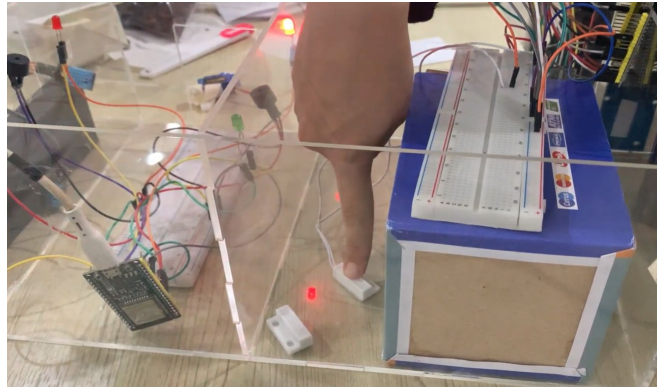
Pengujian dilakukan dengan membaca nilai suhu dan kelembaban dari enam sensor DHT11 secara bersamaan. Hasil pembacaan dikirim ke MQTT broker dan ditampilkan pada dashboard Grafana.

#### b. Pengujian Sensor Getaran



Pengujian dilakukan dengan memberikan getaran fisik pada sensor SW-420. Sensor berhasil mendeteksi getaran dan mengirim nilai “1” ke *topic* bank/vault/vibration. Status pada dashboard menampilkan indikator berubah sesuai kondisi getaran. Alert aktif selama 5 detik sejak getaran terakhir terdeteksi sebelum kembali ke status aman.

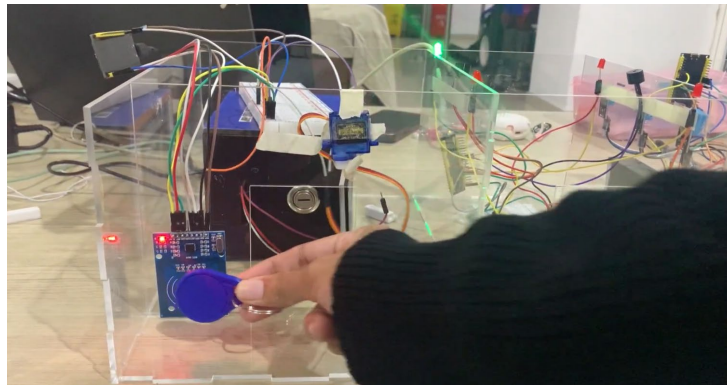
c. Pengujian Sensor Pintu



Pengujian dilakukan dengan membuka dan menutup pintu vault secara manual. Ketika pintu terbuka tanpa otorisasi, sistem mengirim pesan `open_tanpa_akses` ke `topic bank/vault/door` dan dashboard menampilkan status tersebut beserta timestamp kejadian. Ketika pintu tertutup kembali, status kembali normal.

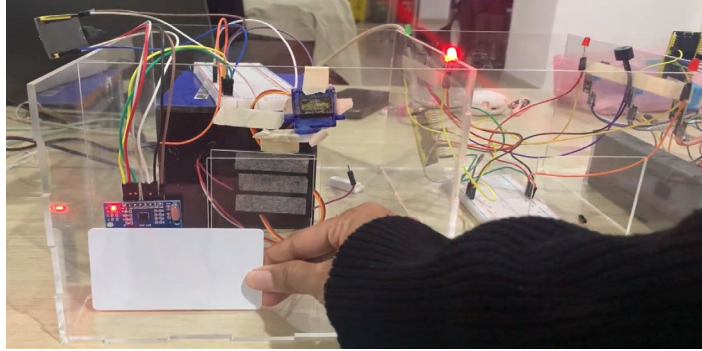
d. Pengujian Sensor RFID

1) Ketika akses sah



Pengujian dilakukan dengan menempelkan kartu RFID yang telah terdaftar pada reader RC522. Sistem membaca UID kartu dan mencocokkan dengan daftar validUID di firmware. Kartu yang cocok menghasilkan respons `granted` pada `topic bank/vault/access`, servo membuka pintu selama 5 detik, LED menyala, dan buzzer berbunyi satu kali sebagai tanda konfirmasi.

2) Ketika akses tidak sah



Pengujian dilakukan dengan menempelkan kartu yang tidak terdaftar. Sistem mengirim pesan denied ke *topic* bank/vault/access dan akses\_illegal ke *topic* bank/vault/alert. LED merah berkedip dua kali dan buzzer berbunyi dua kali sebagai tanda penolakan. Servo tidak bergerak dan pintu tetap terkunci.

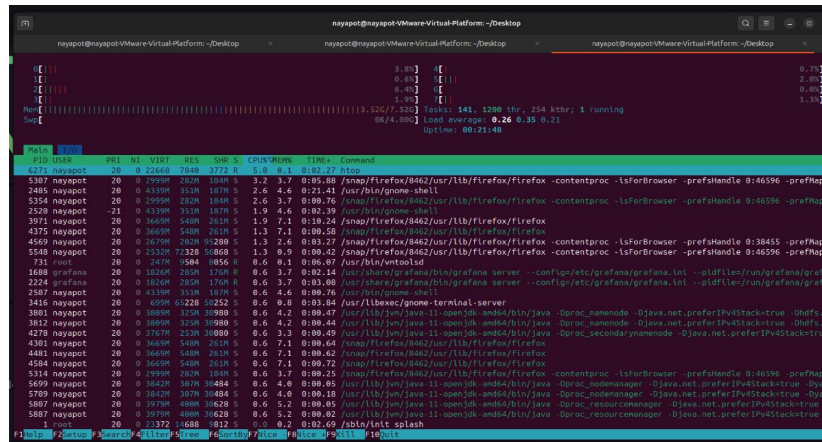
#### 4.3.2 Pengujian Jaringan

```
nayapot@nayapot-VMware-Virtual-Platform: ~/Desktop
nayapot@nayapot-VMware-Virtual-Platform: ~/Desktop$ ping 10.33.109.55
PING 10.33.109.55 (10.33.109.55) 56(84) bytes of data:
64 bytes from 10.33.109.55: icmp_seq=1 ttl=128 time=22.9 ms
64 bytes from 10.33.109.55: icmp_seq=2 ttl=128 time=62.6 ms
64 bytes from 10.33.109.55: icmp_seq=3 ttl=128 time=96.2 ms
64 bytes from 10.33.109.55: icmp_seq=4 ttl=128 time=9.62 ms
64 bytes from 10.33.109.55: icmp_seq=5 ttl=128 time=49.6 ms
64 bytes from 10.33.109.55: icmp_seq=6 ttl=128 time=8.63 ms
^C
--- 10.33.109.55 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 8.625/41.587/96.162/31.478 ms
nayapot@nayapot-VMware-Virtual-Platform: ~/Desktop$
```

Pengujian dilakukan dengan ping dari VM Linux ke MQTT broker di IP 10.33.109.55. Hasilnya menunjukkan 6 paket terkirim dan 6 paket diterima dengan packet loss 0%. Rata-rata latency tercatat 41,587 ms dengan nilai minimum 8,625ms dan maksimum 96,162ms,

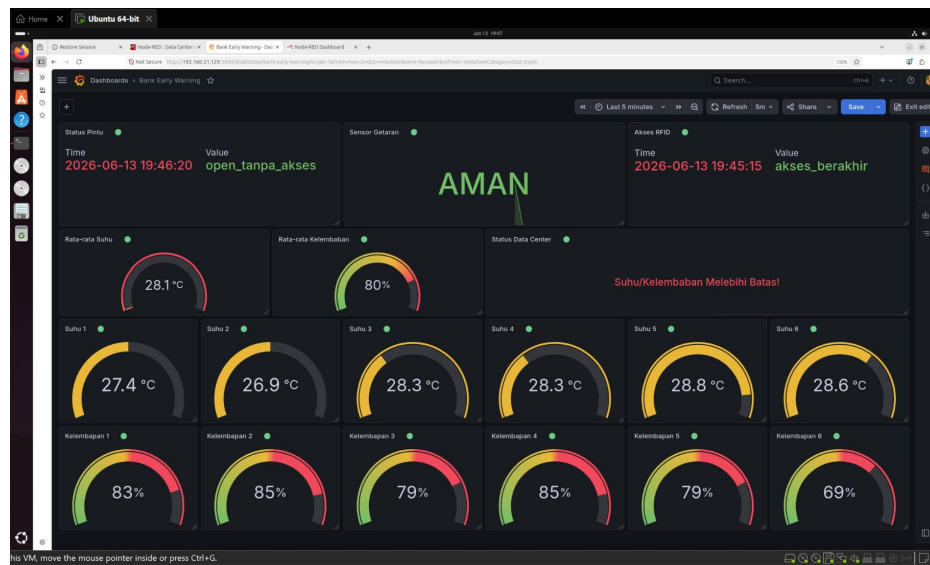
#### 4.3.3 Pengujian Kinerja Aplikasi

- a) Uji Beban CPU & Memory di VM



Pengujian dilakukan menggunakan *tool* htop untuk memantau penggunaan CPU dan memori pada VM saat sistem berjalan penuh. Berdasarkan hasil pengujian, *load average* tercatat 0,26 / 0,35 / 0,21 dengan uptime 21 menit dan 1 proses aktif berjalan. Total memori yang digunakan sekitar 3,52GB dari 7,52GB yang tersedia. Proses utama yang berjalan meliputi Grafana, Node-RED, Hadoop, dan Firefox. Hal tersebut menunjukkan bahwa VM mampu menjalankan seluruh layanan secara bersamaan tanpa terjadi *overload*.

## b) Uji Grafana



Pengujian dilakukan dengan memantau tampilan dashboard Grafana secara real-time. Dashboard berhasil menampilkan seluruh data sensor secara langsung meliputi status pintu, sensor getaran, akses RFID, rata-rata suhu dan kelembaban, serta pembacaan keenam sensor secara individual. Refresh dilakukan setiap 5 menit secara otomatis. Semua panel merespons perubahan data dengan baik tanpa delay yang signifikan.

#### 4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, seluruh komponen utama pada sistem Early Warning Infrastruktur Bank dapat berfungsi sesuai dengan perancangan yang telah dibuat. Sensor DHT11 pada MCU 1 berhasil membaca data suhu dan kelembaban secara real-time serta mengirimkan informasi tersebut ke MQTT Broker melalui jaringan Wi-Fi. Sistem juga mampu menghasilkan status peringatan ketika nilai suhu atau kelembaban melebihi batas yang telah ditentukan. Pada sisi keamanan vault, RFID RC522 berhasil melakukan autentikasi kartu akses, sensor getaran SW420 mampu mendeteksi adanya gangguan fisik, dan sensor pintu MC38 dapat memantau kondisi buka atau tutup pintu dengan baik. Seluruh data keamanan tersebut berhasil dipublikasikan ke MQTT Broker dan diterima oleh perangkat yang berlangganan pada topik terkait.

Dari sisi komunikasi data, penggunaan protokol MQTT menunjukkan kinerja yang baik dalam mendukung pertukaran data antar perangkat IoT. MQTT Broker yang berjalan pada VPS kampus berhasil menerima dan mendistribusikan data dari publisher ke subscriber secara real-time. Hasil pengujian jaringan menunjukkan bahwa proses pengiriman data berlangsung dengan stabil tanpa kehilangan paket yang signifikan sehingga informasi yang dikirim oleh perangkat ESP32 dapat diterima dengan baik oleh Node-RED, Grafana, maupun MCU 3 sebagai perangkat monitoring lokal. Hal ini menunjukkan bahwa arsitektur publish-subscribe yang digunakan mampu mendukung kebutuhan monitoring dan notifikasi pada sistem.

Selain itu, proses visualisasi data menggunakan Grafana dan penyimpanan data menggunakan Apache Hadoop HDFS juga berjalan sesuai dengan yang diharapkan. Dashboard mampu menampilkan informasi suhu, kelembaban, status getaran, kondisi pintu, dan autentikasi RFID secara real-time sehingga memudahkan proses pemantauan kondisi infrastruktur perbankan. Data yang diterima juga berhasil disimpan sebagai arsip historis untuk keperluan backup dan analisis lebih lanjut. Secara keseluruhan, hasil pengujian menunjukkan bahwa sistem yang dikembangkan telah berhasil mengintegrasikan perangkat IoT, protokol MQTT, platform monitoring, dan sistem penyimpanan data dalam satu kesatuan yang mampu mendukung fungsi monitoring serta peringatan dini pada infrastruktur perbankan.

Meskipun sistem telah berhasil berjalan sesuai perancangan, masih terdapat beberapa keterbatasan yang perlu diperhatikan. Kinerja sistem sangat bergantung pada ketersediaan jaringan Wi-Fi dan koneksi ke MQTT Broker, sehingga gangguan jaringan dapat memengaruhi proses pengiriman data secara real-time. Selain itu, implementasi yang dilakukan masih berupa prototype sehingga jumlah perangkat dan cakupan area monitoring masih terbatas. Pada pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan mekanisme redundansi jaringan, notifikasi berbasis aplikasi mobile, serta

integrasi dengan lebih banyak sensor untuk meningkatkan keandalan dan cakupan monitoring infrastruktur perbankan.

## E. PENUTUP

### 5.1 Kesimpulan

Berdasarkan implementasi sistem IoT yang telah dilakukan, dapat ditarik beberapa simpulan sebagai berikut:

1. Integrasi semua sensor berhasil diimplementasikan  
Sistem berhasil mengintegrasikan enam sensor DHT11 yang merepresentasikan kondisi suhu dan kelembaban pada rak server data center. Pembagian sensor dibagi menjadi dua kelompok (tiga pada posisi kiri dan tiga pada posisi kanan). Selain itu, integrasi sensor getaran SW-420, sensor pintu Normally Open, dan modul RFID RC522 pada MCU2 berhasil membentuk sistem deteksi keamanan yang saling melengkapi. Sensor getaran mendeteksi status akses ruangan, dan modul RFID memvalidasi identitas pengguna melalui pembacaan UID kartu menggunakan komunikasi SPI.
2. Dashboard monitoring berhasil menampilkan data secara real-time  
Dashboard pada Grafana berhasil diimplementasikan sebagai pusat visualisasi data dari seluruh sensor dan sistem keamanan. Data suhu, kelembaban, status akses RFID, getaran, dan kondisi pintu brankas ditampilkan secara real-time melalui subscribe ke *topic* MQTT yang relevan, sehingga operator dapat memantau kondisi infrastruktur perbankan kaan saja tanpa harus berada di lokasi fisik perangkat.

### 5.2 Saran

Setelah mengimplementasi sistem IoT, untuk implementasi selanjutnya dapat ditambahkan beberapa saran berikut:

1. Peningkatan spesifikasi sensor  
Disarankan mengganti DHT11 dengan sensor yang memiliki akurasi lebih tinggi seperti DHT22 atau SHT31, mengingat DHT11 memiliki keterbatasan akurasi sebesar  $\pm 2^{\circ}\text{C}$  untuk suhu dan  $\pm 5\%$  untuk kelembaban. Peningkatan ini penting untuk monitoring data center yang membutuhkan presisi lebih ketat, terutama mengingat rentang suhu ideal server yang cukup sempit.
2. Optimalisasi distribusi daaya  
Disarankan menggunakan sumber daya eksternal terpisah untuk komponen yang membutuhkan arus lebih besar seperti motor servo dan buzzer, agar tidak terjadi *drop voltage* pada jalur VCC yang digunakan bersama. Selain itu, penambahan kapasitor pada jalur daya servo dapat membantu menstabilkan tegangan saat servo bergerak sehingga tidak mengganggu kinerja komponen lain yang terhubung pada MCU yang sama.



## DAFTAR PUSTAKA

- [1] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, “Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications,” *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.
- [3] N. Komninos, E. Philippou, and A. Pitsillides, “Survey in Smart Grid and Smart Home Security,” *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 1933–1954, 2014.
- [4] OASIS, *MQTT Version 3.1.1 Plus Errata 01*, OASIS Standard, 2015.
- [5] Otoritas Jasa Keuangan, *Panduan Resiliensi Digital bagi Bank Umum*, Jakarta, Indonesia, 2024.
- [6] K. Ashton, “That ‘Internet of Things’ Thing,” *RFID Journal*, 2009.
- [7] Uptime Institute, *Data Center Temperature and Cooling Best Practices*, 2023.
- [8] Espressif Systems, *ESP32 Technical Reference Manual*, 2024.
- [9] A. Dennis, *Electronic Security Systems*, Boston: Cengage Learning, 2016.
- [10] D. Conway-Jones and N. O’Leary, *Node-RED Documentation*, IBM, 2024.
- [11] Grafana Labs, *Grafana Documentation*, 2024.
- [12] R. Buyya, J. Broberg, and A. Goscinski, *Cloud Computing: Principles and Paradigms*, Hoboken, NJ: Wiley, 2011.
- [13] D. Bandyopadhyay and J. Sen, “Internet of Things: Applications and Challenges in Technology and Standardization,” *Wireless Personal Communications*, vol. 58, no. 1, pp. 49–69, 2011.
- [14] A. Basher, “Global Early Warning Systems for Natural Hazards,” *Philosophical Transactions of the Royal Society A*, vol. 364, no. 1845, pp. 2167–2182, 2006.



```

DATA_CENTER_MQTT.ino
83
84 void setup() {
85   Serial.begin(115200);
86   delay(1000);
87
88   dht1.begin(); dht2.begin(); dht3.begin();
89   dht4.begin(); dht5.begin(); dht6.begin();
90
91   WiFi.begin(ssid, password);
92   while (WiFi.status() != WL_CONNECTED) delay(500);
93   Serial.println("\n🟢 WiFi connected");
94
95   client.setServer(mqtt_server, mqtt_port);
96   while (!client.connected()) {
97     if (client.connect(mqtt_client_id)) Serial.println("🟢 MQTT connected");
98     else delay(5000);
99   }
100
101   Serial.println("MCU1 - Data Center Ready");
102 }
103
104 void loop() {
105   if (!client.connected()) client.connect(mqtt_client_id);
106   client.loop();
107
108   float suhu[6] = { bacaSuhu(dht1), bacaSuhu(dht2), bacaSuhu(dht3),
109                   bacaSuhu(dht4), bacaSuhu(dht5), bacaSuhu(dht6) };
110   float hum[6] = { bacaHumidity(dht1), bacaHumidity(dht2), bacaHumidity(dht3),
111                   bacaHumidity(dht4), bacaHumidity(dht5), bacaHumidity(dht6) };
112
113   bool alert = cekAlert(suhu, hum);
114   float avgTemp = hitungRataRata(suhu);
115   float avgHum = hitungRataRata(hum);
116
117   client.publish(topic_alert, alert ? "1" : "0");
118   client.publish(topic_temperature, String(avgTemp).c_str());
119   client.publish(topic_humidity, String(avgHum).c_str());
120
121   client.publish(topic_suhu1, String(suhu[0]).c_str());
122   client.publish(topic_hum1, String(hum[0]).c_str());
123   client.publish(topic_suhu2, String(suhu[1]).c_str());
124   client.publish(topic_hum2, String(hum[1]).c_str());
125   client.publish(topic_suhu3, String(suhu[2]).c_str());
126   client.publish(topic_hum3, String(hum[2]).c_str());
127   client.publish(topic_suhu4, String(suhu[3]).c_str());
128   client.publish(topic_hum4, String(hum[3]).c_str());
129   client.publish(topic_suhu5, String(suhu[4]).c_str());
130   client.publish(topic_hum5, String(hum[4]).c_str());
131   client.publish(topic_suhu6, String(suhu[5]).c_str());
132   client.publish(topic_hum6, String(hum[5]).c_str());
133
134   if (millis() - lastHeartbeat > heartbeatInterval) {
135     client.publish(topic_heartbeat, "1");
136     lastHeartbeat = millis();
137   }
138
139   Serial.print("DC Alert: ");
140   Serial.println(alert ? "YES" : "NO");
141
142   delay(3000);
143 }

```

- Vault

```

VAULT_MQTT.ino
6 #include <SPI.h>
7 #include <MFRC522.h>
8 #include <WiFi.h>
9 #include <PubSubClient.h>
10 #include <ESP32Servo.h>
11
12 // ===== PIN DEFINITION =====
13 #define PIN_VIBRATION 34
14 #define PIN_DOOR 35
15 #define SS_PIN 4
16 #define RST_PIN 22
17 #define SCK_PIN 18
18 #define MOSI_PIN 23
19 #define MISO_PIN 19
20 #define SERVO_PIN 13
21
22 // ===== SERVO =====
23 Servo myServo;
24 const int SERVO_TERTUTUP = 0;
25 const int SERVO_TERBUKA = 90;
26
27 // ===== WIFI & MQTT =====
28 const char* ssid = "bundouwo";
29 const char* password = "0987654321";
30 const char* mqtt_server = "10.33.109.55";
31 const int mqtt_port = 1883;
32 const char* mqtt_client = "MCU2_Vault";
33
34 // ===== MQTT TOPICS =====
35 const char* topic_vibration = "bank/vault/vibration";
36 const char* topic_door = "bank/vault/door";
37 const char* topic_rfid = "bank/vault/access";
38 const char* topic_alert = "bank/vault/alert";
39
40 WiFiClient espClient;
41 PubSubClient mqttClient(espClient);
42 MFRC522 mfrc522(SS_PIN, RST_PIN);
43

```

```

VAULT_MQTT.ino
81
82 void connectWiFi() {
83   WiFi.begin(ssid, password);
84   while (WiFi.status() != WL_CONNECTED) delay(500);
85   Serial.println("WiFi connected");
86 }
87
88 void connectMQTT() {
89   while (!mqttClient.connected()) {
90     if (mqttClient.connect(mqtt_client)) Serial.println("MQTT connected");
91     else delay(5000);
92   }
93 }
94
95 void publishMQTT(const char* topic, String payload) {
96   if (mqttClient.connected()) {
97     mqttClient.publish(topic, payload.c_str());
98   }
99 }
100
101 bool isCardValid() {
102   for (int i = 0; i < jumlahKartuValid; i++) {
103     bool match = true;
104     for (byte j = 0; j < 4; j++) {
105       if (wfrs22.uid.uidbyte[j] != validUID[i][j]) match = false;
106     }
107     if (match) return true;
108   }
109   return false;
110 }
111
112 void setup() {
113   Serial.begin(115200);
114   delay(500);
115   myServo.attach(SERVO_PIN);
116   myServo.write(SERVO_TERTUTUP);
117   SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
118   wfrs22.PCB_Init();
119 }
120
VAULT_MQTT.ino
43
44 // UID kartu valid
45 byte validUID[][4] = {{0xF6, 0x10, 0xEC, 0x05}};
46 int jumlahKartuValid = 1;
47
48 // State variables
49 bool aksesAman = false;
50 unsigned long lastAccessTime = 0;
51 const unsigned long accessDuration = 5000;
52 bool illegalCardScanned = false;
53 unsigned long lastIllegalWarningTime = 0;
54 const unsigned long illegalWarningCooldown = 3000;
55 unsigned long lastScanTime = 0;
56 const unsigned long scanCooldown = 2000;
57
58 // ===== GETARAN DENGAN TIMER 5 DETIK =====
59 bool vibrationAlertActive = false;
60 unsigned long lastVibrationTriggerTime = 0;
61 const unsigned long vibrationAlertDuration = 5000;
62
63 // ===== SENSOR PINTU DENGAN DEBOUNCE =====
64 bool lastDoorState = false;
65 bool lastPublishedDoorState = false; // Status terakhir yang dipublish
66 unsigned long lastDebounceTime = 0;
67 const unsigned long debounceDelay = 50; // 50ms debounce
68
69 // Anti-spam illegal warning
70 bool illegalWarningPrinted = false;
71
72 void bukaPintu() {
73   Serial.println("🔓 Servo: Membuka pintu...");
74   myServo.write(SERVO_TERBUKA);
75 }
76
77 void tutupPintu() {
78   Serial.println("🔒 Servo: Menutup pintu...");
79   myServo.write(SERVO_TERTUTUP);
80 }

```

```

VAULT_MQTTino
121
122 pinMode(PIN_VIBRATION, INPUT);
123 pinMode(PIN_DOOR, INPUT_PULLUP);
124
125 // Baca status awal pintu
126 lastDoorState = (digitalRead(PIN_DOOR) == LOW);
127 lastPublishedDoorState = lastDoorState;
128
129 connectWifi();
130 mqttclient.setServer(mqtt_server, mqtt_port);
131 connectMQTT();
132
133 Serial.println("=====");
134 Serial.println(" MCU2 - Vault Ready");
135 Serial.println("=====");
136 Serial.println("Getaran: Alert 5 detik");
137 Serial.println("=====");
138 }
139
140 void loop() {
141   if (WiFi.status() != WL_CONNECTED) connectWifi();
142   if (!mqttclient.connected()) connectMQTT();
143   mqttclient.loop();
144
145   unsigned long now = millis();
146
147   // ===== RFID =====
148   if (mfrc522.PICC_IsNewCardPresent() && mfrc522.PICC_ReadCardSerial()) {
149     if (now - lastScanTime >= scanCooldown) {
150       lastScanTime = now;
151       String uid = "";
152       for (byte i = 0; i < mfrc522.uid.size; i++) {
153         if (mfrc522.uid.uidByte[i] < 0x10) uid += "0";
154         uid += String(mfrc522.uid.uidByte[i], HEX);
155       }
156       Serial.print("UID: "); Serial.println(uid);
157
158       if (isCardValid()) {
159         Serial.println("✅ AKSES DITERIMA!");
160         aksesAman = true;
161         lastAccessTime = now;
162         illegalCardsScanned = false;
163         bukaPintu();
164         publishMQTT(topic_rfid, "granted");
165       } else {
166         Serial.println("❌ AKSES DITOLAK!");
167         illegalCardsScanned = true;
168         lastIllegalWarningTime = now;
169         publishMQTT(topic_rfid, "denied");
170         publishMQTT(topic_alert, "akses_illegal");
171       }
172     }
173     mfrc522.PICC_HaltA();
174     mfrc522.PCD_StopCrypto1();
175   }
176
177   // ===== Update akses aman =====
178   if (aksesAman && (now - lastAccessTime > accessDuration)) {
179     aksesAman = false;
180     tutupPintu();
181     publishMQTT(topic_rfid, "akses_berakhir");
182   }
183
184   // ===== SENSOR GETAR (ALERT 5 DETIK) =====
185   bool currentVibration = (digitalRead(PIN_VIBRATION) == HIGH);
186
187   if (currentVibration) {
188     lastVibrationTriggerTime = now;
189     if (!vibrationAlertActive) {
190       vibrationAlertActive = true;
191       Serial.println("🚨 GETARAN TERDETEKSI! (Alert 5 detik)");
192       publishMQTT(topic_vibration, "1");
193     }
194   }
195 }

```

```

VAULT_MQTT.ino
195
196 bool alertActive = (now - lastVibrationTriggerTime < vibrationAlertDuration);
197 if (alertActive && vibrationAlertActive) {
198     vibrationAlertActive = false;
199     Serial.println("🔔 Alert getaran berakhir (5 detik selesai)");
200     publishMQTT(topic_vibration, "0");
201 }
202
203 // ===== SENSOR PINTU DENGAN DEBOUNCE (TIDAK SPAM) =====
204 bool rawDoorState = (digitalRead(PIN_DOOR) == LOW); // LOW = terbuka
205
206 // Debounce: hanya berubah jika status stabil selama debounceDelay
207 if (rawDoorState != lastDoorState) {
208     lastDebounceTime = now;
209     lastDoorState = rawDoorState;
210 }
211
212 // Jika status stabil dan berbeda dengan yang sudah dipublish
213 if ((now - lastDebounceTime) > debounceDelay) {
214     if (rawDoorState != lastPublishedDoorState) {
215         lastPublishedDoorState = rawDoorState;
216
217         if (rawDoorState) {
218             // Pintu terbuka
219             if (aksesAman) {
220                 Serial.println("🔓 Pintu Terbuka - Akses Sah");
221                 publishMQTT(topic_door, "open_sah");
222             } else {
223                 Serial.println("⚠️ WARNING! Pintu Terbuka Tanpa Otorisasi!");
224                 publishMQTT(topic_door, "open_tanpa_aksas");
225                 publishMQTT(topic_alert, "pintu_tanpa_otorisasi");
226             }
227         } else {
228             // Pintu tertutup
229             Serial.println("🔒 Pintu Tertutup");
230             publishMQTT(topic_door, "closed");
231         }
232     }
233 }
234
235 // ===== Warning Illegal (Anti-Spam) =====
236 if (illegalCardScanned && rawDoorState && aksesAman) {
237     if (!illegalWarningPrinted) {
238         Serial.println("🚫 PERINGATAN! Akses Illegal - Pintu Seharusnya Tertutup!");
239         publishMQTT(topic_alert, "akses_illegal_pintu_terbuka");
240         illegalWarningPrinted = true;
241     }
242 } else {
243     illegalWarningPrinted = false;
244 }
245
246 delay(10);
247 }

```

- Monitoring

```

MONITORING_MQTT.ino
12 #include <WiFi.h>
13 #include <PubSubClient.h>
14
15 // ===== PIN DEFINISI =====
16 #define PIN_LED_MERAH_GETARAN 21
17 #define PIN_LED_MERAH_DC 16
18 #define PIN_BUZZER_DC 17
19 #define PIN_LED_HIJAU_VAULT 32
20 #define PIN_LED_MERAH_VAULT 33
21 #define PIN_BUZZER_VAULT 25
22
23 // ===== KONFIGURASI =====
24 const char* ssid = "bundowmo";
25 const char* password = "0987654321";
26 const char* mqtt_server = "10.33.109.55";
27 const int mqtt_port = 1883;
28
29 WiFiClient espClient;
30 PubSubClient client(espClient);
31
32 // ===== STATUS =====
33 bool alertDC = false;
34 bool alertVibration = false;
35 bool pintuTerbukaTanpaAkses = false;
36 bool aksesSah = false;
37 unsigned long aksesSahStartTime = 0;
38 const unsigned long aksesDurasi = 5000;
39
40 unsigned long aksesDitolakStartTime = 0;
41 bool aksesDitolak = false;
42 const unsigned long aksesDitolakDurasi = 3000;
43
44 unsigned long alertDCStartTime = 0;
45 unsigned long alertVibrationStartTime = 0;
46 bool alertDCActive = false;
47 bool alertVibrationActive = false;
48 const unsigned long alertDuration = 5000;
49

```

```

MONITORING_MQTT.ino
49
50 unsigned long lastBlinkTimeDC = 0;
51 unsigned long lastBlinkTimeVault = 0;
52 bool blinkStateDC = false;
53 bool blinkStateVault = false;
54
55 void setup() {
56   Serial.begin(115200);
57   delay(500);
58
59   pinMode(PIN_LED_MERAH_GETARAN, OUTPUT);
60   pinMode(PIN_LED_MERAH_DC, OUTPUT);
61   pinMode(PIN_BUZZER_DC, OUTPUT);
62   pinMode(PIN_LED_HIJAU_VAULT, OUTPUT);
63   pinMode(PIN_LED_MERAH_VAULT, OUTPUT);
64   pinMode(PIN_BUZZER_VAULT, OUTPUT);
65
66   // Matikan semua
67   digitalWrite(PIN_LED_MERAH_GETARAN, LOW);
68   digitalWrite(PIN_LED_MERAH_DC, LOW);
69   digitalWrite(PIN_BUZZER_DC, LOW);
70   digitalWrite(PIN_LED_HIJAU_VAULT, LOW);
71   digitalWrite(PIN_LED_MERAH_VAULT, LOW);
72   digitalWrite(PIN_BUZZER_VAULT, LOW);
73
74   WiFi.begin(ssid, password);
75   while (WiFi.status() != WL_CONNECTED) delay(500);
76   Serial.println("WiFi connected");
77
78   client.setServer(mqtt_server, mqtt_port);
79   client.setCallback(callback);
80   connectMQTT();
81
82   Serial.println("MCU3 - Monitoring Ready");
83   Serial.println("G4 = LED Merah Getaran Vault");
84   Serial.println("G16 = LED Merah DC Alert");
85   Serial.println("G21 = LED Hijau Vault (Akses Sah)");
86   Serial.println("G22 = LED Merah Vault (Pintu Tanpa Akses)");
87 }

```

```

MONITORING_MQTT.ino
88
89 void connectMQTT() {
90   while (!client.connected()) {
91     if (client.connect("MCU3_Monitor")) {
92       client.subscribe("bank/dc/alert");
93       client.subscribe("bank/vault/alert");
94       client.subscribe("bank/vault/vibration");
95       client.subscribe("bank/vault/access");
96       client.subscribe("bank/vault/door");
97       Serial.println("MQTT connected & subscribed");
98     } else delay(5000);
99   }
100 }
101
102 void callback(char* topic, byte* payload, unsigned int length) {
103   String message;
104   for (int i = 0; i < length; i++) message += (char)payload[i];
105
106   Serial.print("Topic: ");
107   Serial.print(topic);
108   Serial.print(" | Msg: ");
109   Serial.println(message);
110
111   String topicStr = String(topic);
112
113   // ===== DC ALERT =====
114   if (topicStr == "bank/dc/alert") {
115     if (message == "1") {
116       alertDC = true;
117       alertDCActive = true;
118       alertDCStartTime = millis();
119       Serial.println("DC ALERT!");
120     } else {
121       alertDC = false;
122     }
123   }

```



```

MONITORING_MQTT.ino
124
125 // ===== GETARAN (LED di G4) =====
126 if (topicStr == "bank/vault/vibration" && message == "1") {
127   alertVibration = true;
128   alertVibrationActive = true;
129   alertVibrationStartTime = millis();
130   Serial.println(" 🚨 GETARAN! (LED Merah G4 menyala)");
131 }
132
133 // ===== AKSES RFID =====
134 if (topicStr == "bank/vault/access") {
135   if (message == "granted") {
136     aksesSah = true;
137     aksesSahStartTime = millis();
138     aksesDitolak = false;
139     Serial.println(" ✅ AKSES SAH");
140     digitalWrite(PIN_LED_HIJAU_VAULT, HIGH);
141     delay(200);
142     digitalWrite(PIN_LED_HIJAU_VAULT, LOW);
143     digitalWrite(PIN_BUZZER_VAULT, HIGH);
144     delay(200);
145     digitalWrite(PIN_BUZZER_VAULT, LOW);
146   }
147   else if (message == "denied") {
148     aksesDitolak = true;
149     aksesDitolakStartTime = millis();
150     aksesSah = false;
151     Serial.println(" ❌ AKSES DITOLAK");
152
153     for (int i = 0; i < 2; i++) {
154       digitalWrite(PIN_LED_MERAH_VAULT, HIGH);
155       digitalWrite(PIN_BUZZER_VAULT, HIGH);
156       delay(200);
157       digitalWrite(PIN_LED_MERAH_VAULT, LOW);
158       digitalWrite(PIN_BUZZER_VAULT, LOW);
159       delay(200);
160     }
161   }
162 }
163
MONITORING_MQTT.ino
164 // ===== PINTU =====
165 if (topicStr == "bank/vault/door") {
166   if (message == "open_tanpa_akses") {
167     pintuTerbukaTanpaAkses = true;
168     Serial.println(" 🚪 PINTU TERBUKA TANPA AKSES!");
169   } else if (message == "open_sah") {
170     Serial.println(" 🚪 PINTU TERBUKA (Akses sah)");
171     digitalWrite(PIN_LED_HIJAU_VAULT, HIGH);
172     digitalWrite(PIN_BUZZER_VAULT, HIGH);
173     delay(300);
174     digitalWrite(PIN_BUZZER_VAULT, LOW);
175   } else if (message == "closed") {
176     pintuTerbukaTanpaAkses = false;
177     digitalWrite(PIN_LED_HIJAU_VAULT, LOW);
178     digitalWrite(PIN_LED_MERAH_VAULT, LOW);
179     digitalWrite(PIN_BUZZER_VAULT, LOW);
180   }
181 }
182 }
183
184 void loop() {
185   if (!client.connected()) connectMQTT();
186   client.loop();
187
188   unsigned long now = millis();
189
190   // ===== RESET AKSES DITOLAK =====
191   if (aksesDitolak && (now - aksesDitolakStartTime > aksesDitolakDurasi)) {
192     aksesDitolak = false;
193     digitalWrite(PIN_LED_MERAH_VAULT, LOW);
194     Serial.println(" ✅ Status akses ditolak berakhir");
195   }
196
197   // ===== UPDATE AKSES SAH =====
198   if (aksesSah && (now - aksesSahStartTime > aksesDurasi)) {
199     aksesSah = false;
200     digitalWrite(PIN_LED_HIJAU_VAULT, LOW);
201     Serial.println(" 🕒 Masa akses sah berakhir");
202   }

```



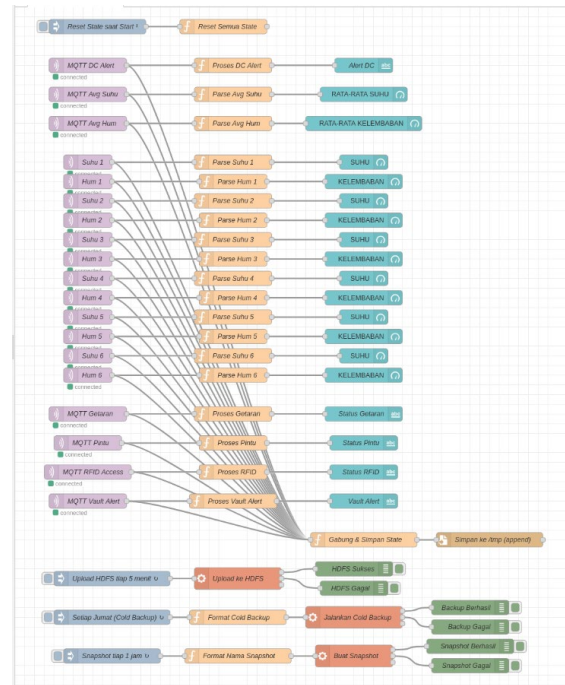
```

203 MONITORING_MQTT.ino
204 // ===== ALERT DC (LED Merah G16 + Buzzer G17) =====
205 if (alertDCActive) {
206     if (now - alertDCStartTime < alertDuration) {
207         if (now - lastBlinkTimeDC > 100) {
208             blinkStateDC = !blinkStateDC;
209             digitalWrite(PIN_LED_MERAH_DC, blinkStateDC);
210             digitalWrite(PIN_BUZZER_DC, blinkStateDC);
211             lastBlinkTimeDC = now;
212         }
213     } else {
214         alertDCActive = false;
215         digitalWrite(PIN_LED_MERAH_DC, LOW);
216         digitalWrite(PIN_BUZZER_DC, LOW);
217         Serial.println("✅ DC kembali normal");
218     }
219 }
220
221 // ===== ALERT GETARAN (LED Merah G4 + Buzzer G23) =====
222 if (alertVibrationActive) {
223     if (now - alertVibrationStartTime < alertDuration) {
224         if (now - lastBlinkTimeVault > 100) {
225             blinkStateVault = !blinkStateVault;
226             digitalWrite(PIN_LED_MERAH_GETARAN, blinkStateVault); // LED Getaran di G4
227             digitalWrite(PIN_BUZZER_VAULT, blinkStateVault);
228             lastBlinkTimeVault = now;
229         }
230     } else {
231         alertVibrationActive = false;
232         digitalWrite(PIN_LED_MERAH_GETARAN, LOW);
233         digitalWrite(PIN_BUZZER_VAULT, LOW);
234         Serial.println("✅ Getaran selesai");
235     }
236 }
237
238 // ===== PINTU TERBUKA TANPA AKSES (LED Merah G22 + Buzzer G23) =====
239 if (pintuTerbukaTanpaAkses && !aksesSah) {
240     if (now - lastBlinkTimeVault > 500) {
241         blinkStateVault = !blinkStateVault;
242         digitalWrite(PIN_LED_MERAH_VAULT, blinkStateVault);
243         digitalWrite(PIN_BUZZER_VAULT, blinkStateVault);
244         lastBlinkTimeVault = now;
245     }
246 }
247
248 // ===== AKSES SAH + PINTU TERBUKA =====
249 if (aksesSah && pintuTerbukaTanpaAkses) {
250     digitalWrite(PIN_LED_HIJAU_VAULT, HIGH);
251 }
252
253 // ===== PASTIKAN LED MATI JIKA TIDAK ADA STATUS =====
254 if (!aksesDitolak && !alertVibrationActive && !pintuTerbukaTanpaAkses && !aksesSah) {
255     digitalWrite(PIN_LED_MERAH_GETARAN, LOW);
256     digitalWrite(PIN_LED_MERAH_VAULT, LOW);
257     digitalWrite(PIN_LED_HIJAU_VAULT, LOW);
258     digitalWrite(PIN_BUZZER_VAULT, LOW);
259 }
260
261 delay(10);
262 }

```

## Lampiran 2. Gambar pendukung

- Flow Node-RED



## Lampiran 3. Dokumentasi prototype

